CS151 Intro to Data Structures

Trees

Stack review

```
insert(x):
    S1.push(x)


delete():
    if S2 is empty:
        while S1 is not empty:
            S2.push(S1.pop())
    return S2.pop()
```

Consider a queue implemented using two stacks S1 and S2 (as in HW3). Starting from an empty queue, we perform n inserts and m deletes (m < n), interleaved in some order.

	min possible	max possible
Total `push` operations		
Total `pop` operations		

Warmup: Binary Search Trees

Write `boolean isBST(Node root)` that returns `true` if the binary tree is a valid BST.

Warmup 2: Binary Search Trees

Starting from an empty BST, insert these keys in order:

50, 30, 70, 20, 40, 60, 80, 35

(a) Draw the resulting tree.
(b) List the nodes you visit when searching for 35. Then do the same for 45.
(c) Searching takes at most how many comparisons in this tree? What property of the tree determines that?
(d) Insert the same eight keys in a different order so that search is as slow as possible. What's the Big O of search in that case?

Announcements

- HW5 released
  - due next Friday Nov 6th
- Office hours today 4-5pm

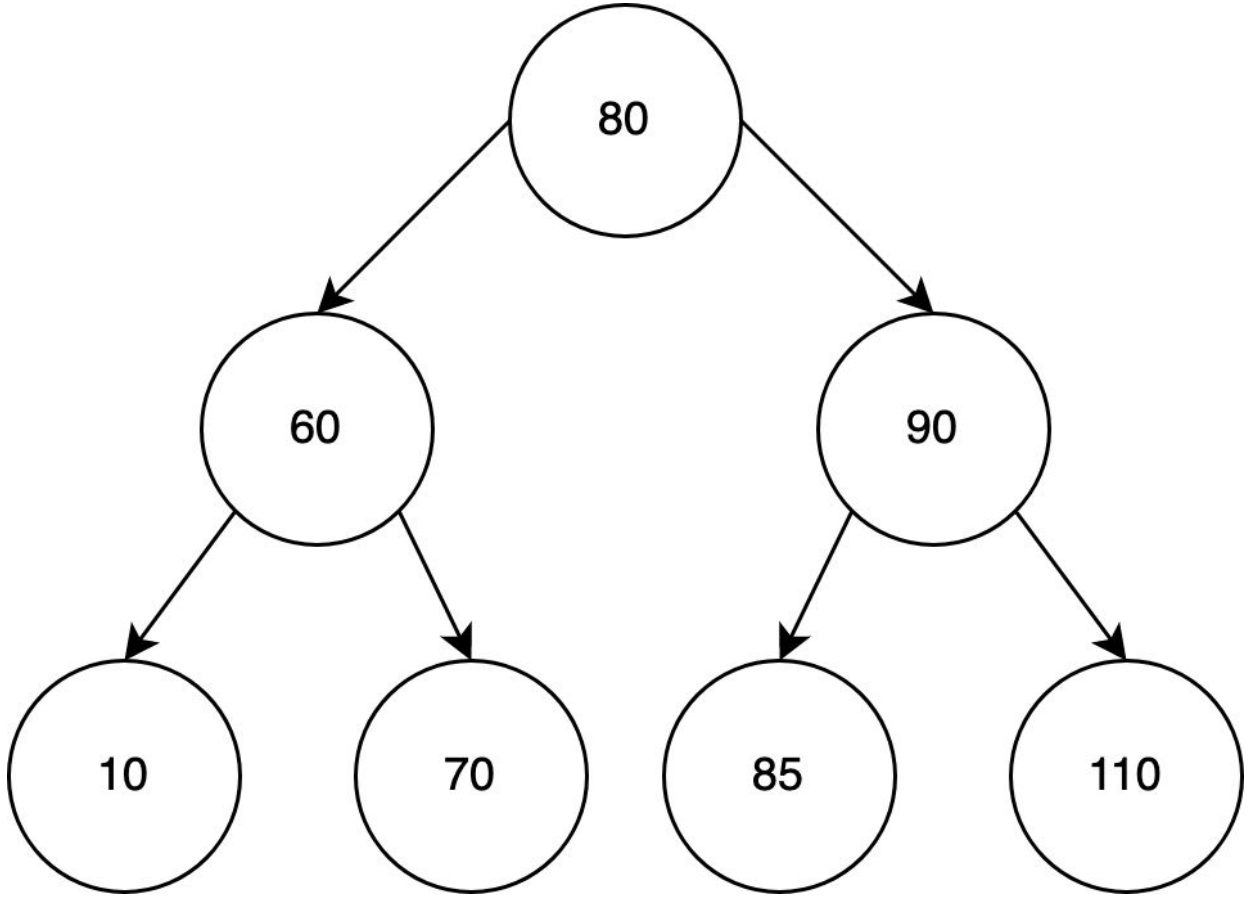
Outline

- Trees:
  - Review: Binary Search Trees
    - Inserting
    - Searching
  - New: deletions
  - New: traversal

Trees: Nodes

```
class Node {
    int key;
    Node left;
    Node right;

    public Node(int item) {
        key = item;
        left = null;
        right = null;
    }
}
```



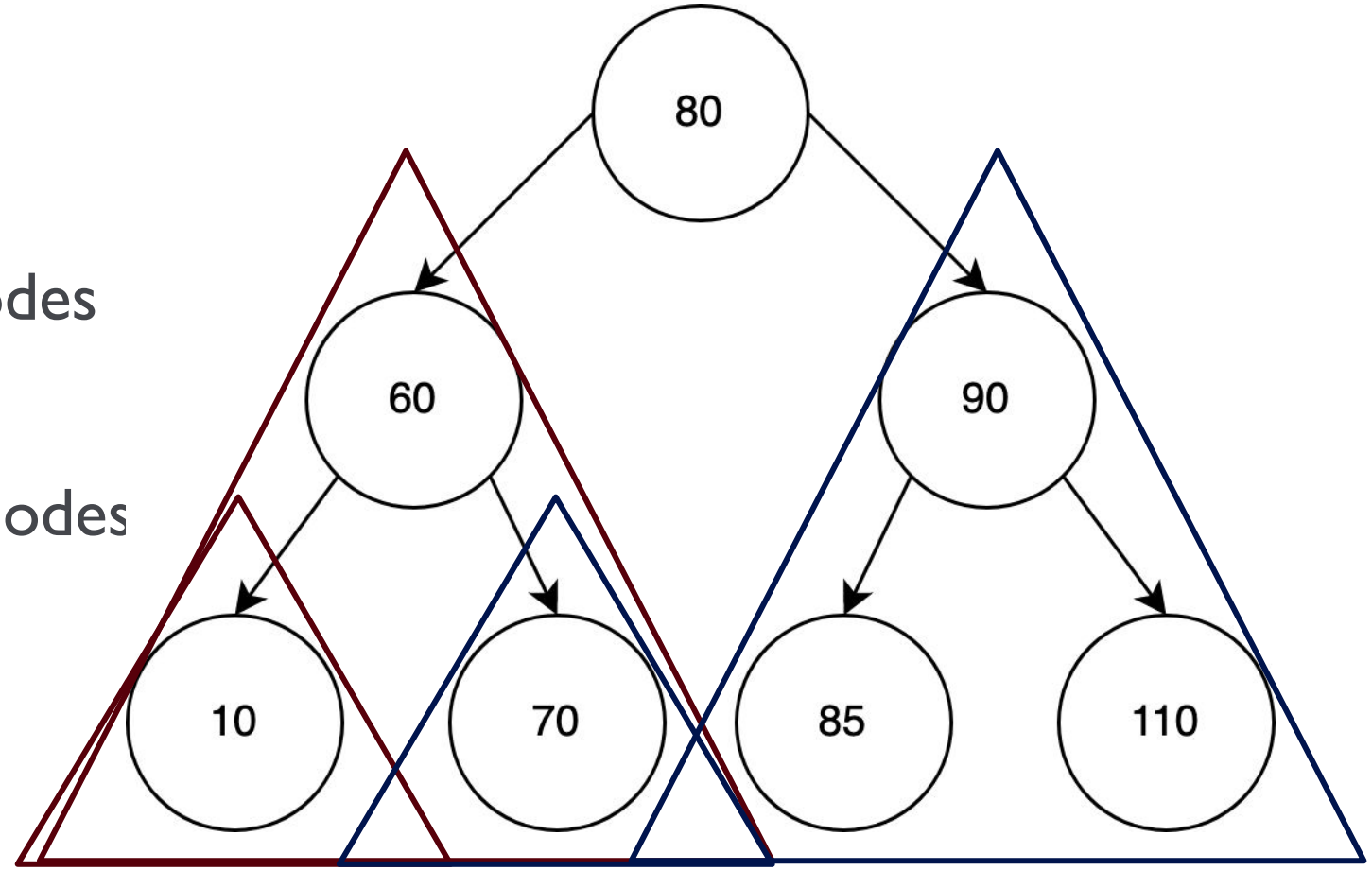
Binary Search Trees

Binary Search Trees

Definition:

At **each node** with value **k**

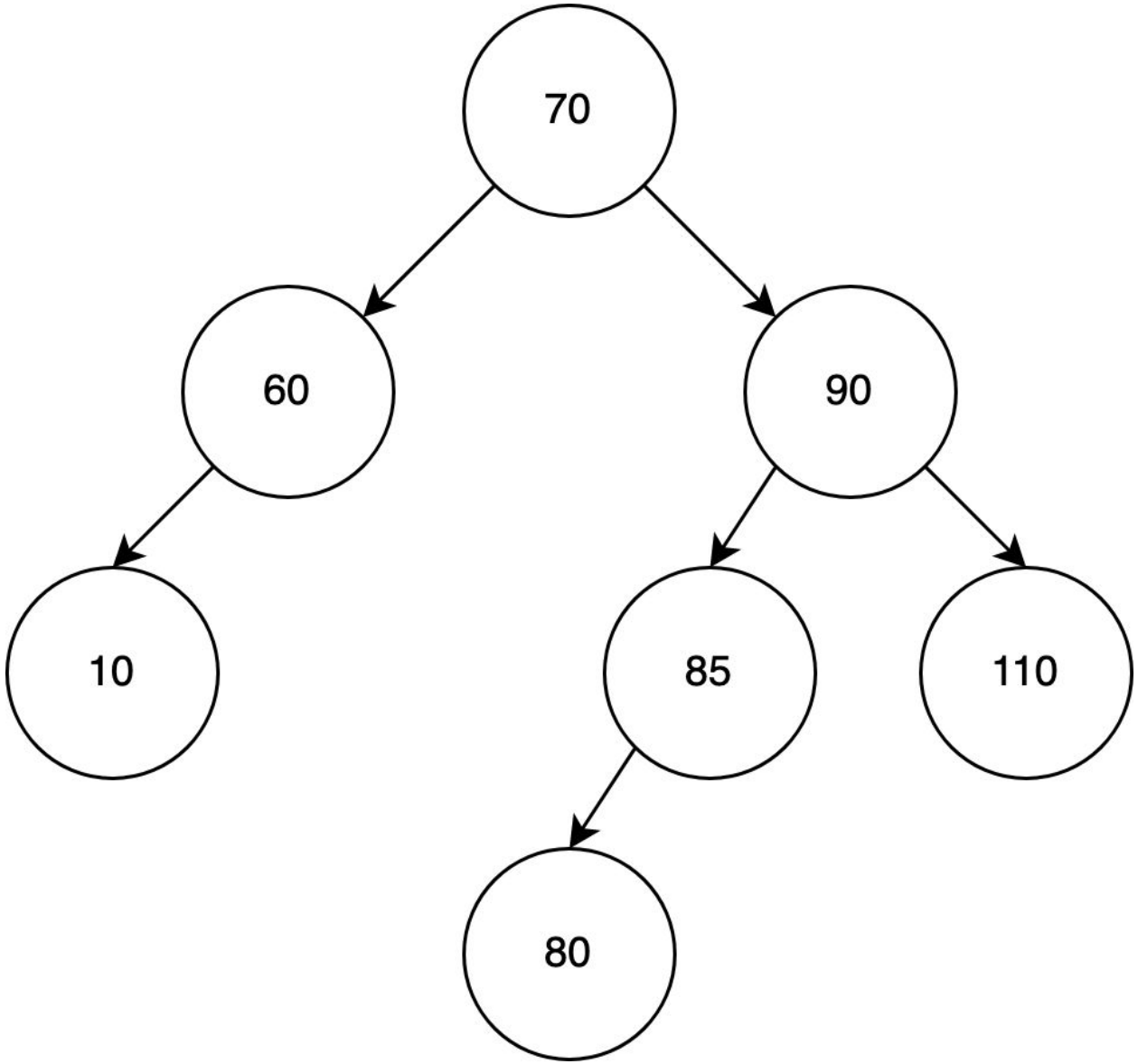
- Left subtree contains only nodes with value **lesser** than **k**
- Right subtree contains only nodes with value **greater** than **k**
- Both subtrees are a **binary search tree**



Exercise One: Binary Search Trees

Is this a binary search tree?

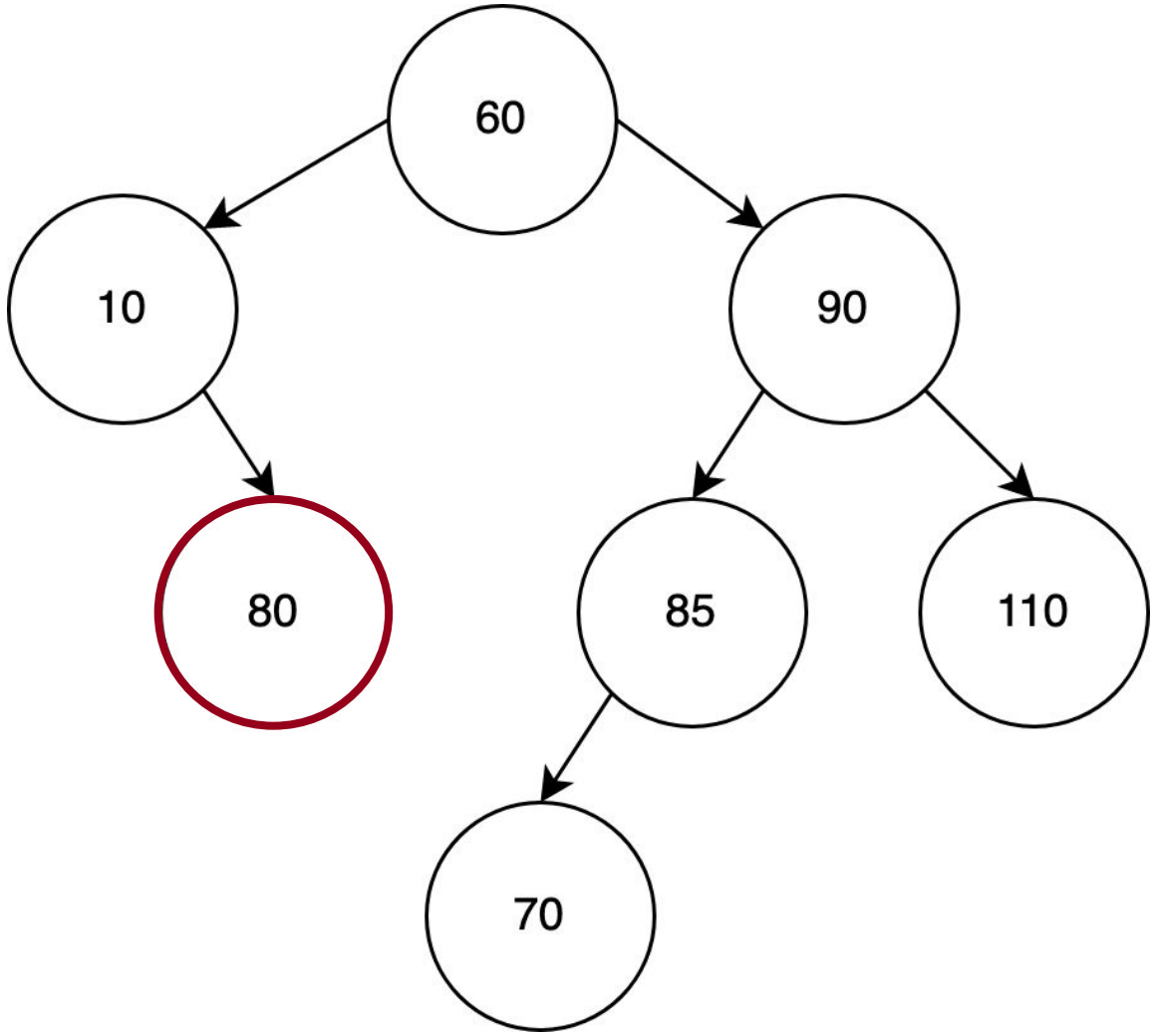




Exercise One: Binary Search Trees

Is this a binary search tree?

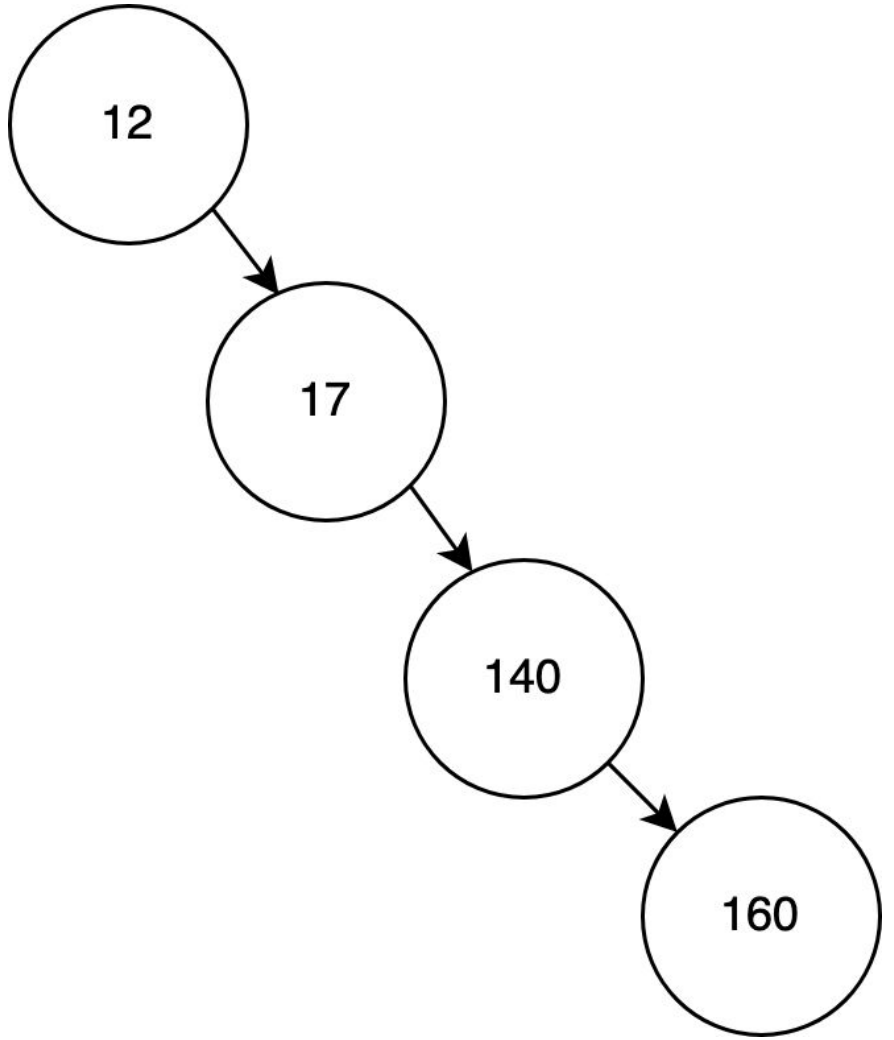




Exercise One: Binary Search Trees

Is this a binary search tree?





Today's Lecture

1. Binary Search Trees
2. **Search**
3. Insertion
4. Removal
5. Summary

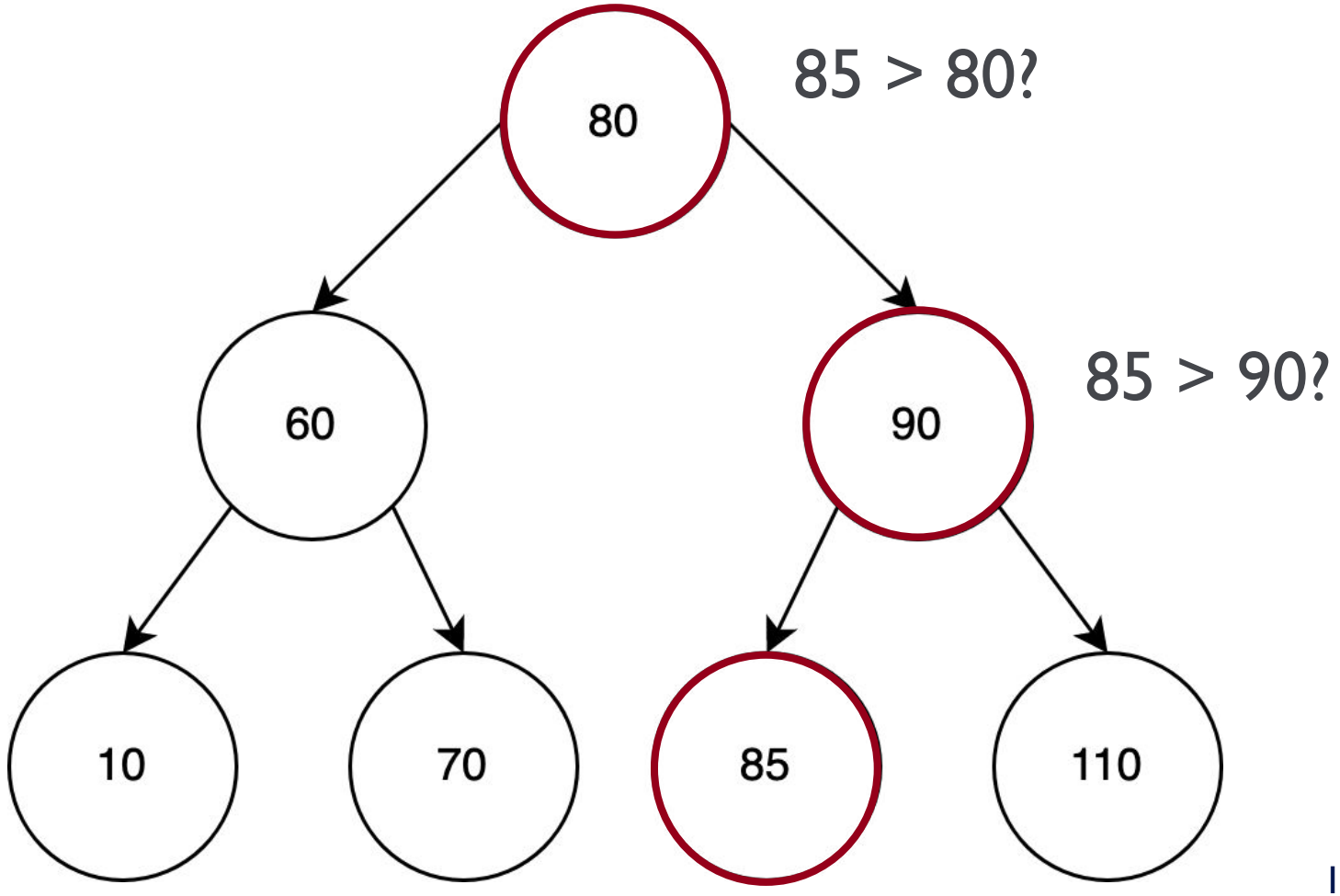
Binary Search Trees: Efficient Search

Goal: Report if a value exists in the tree

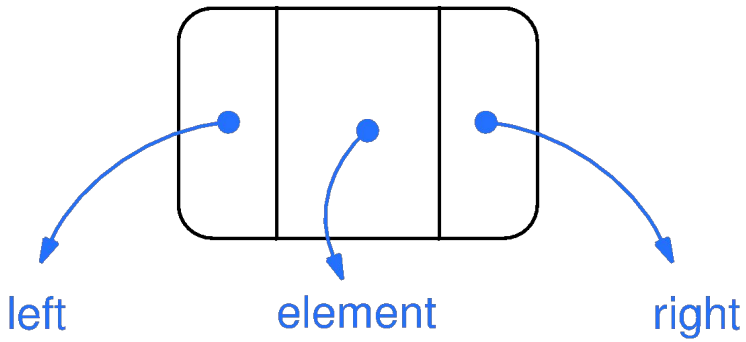
Target: 85

if **target** > **k**:
    Move right
else:
    Move Left

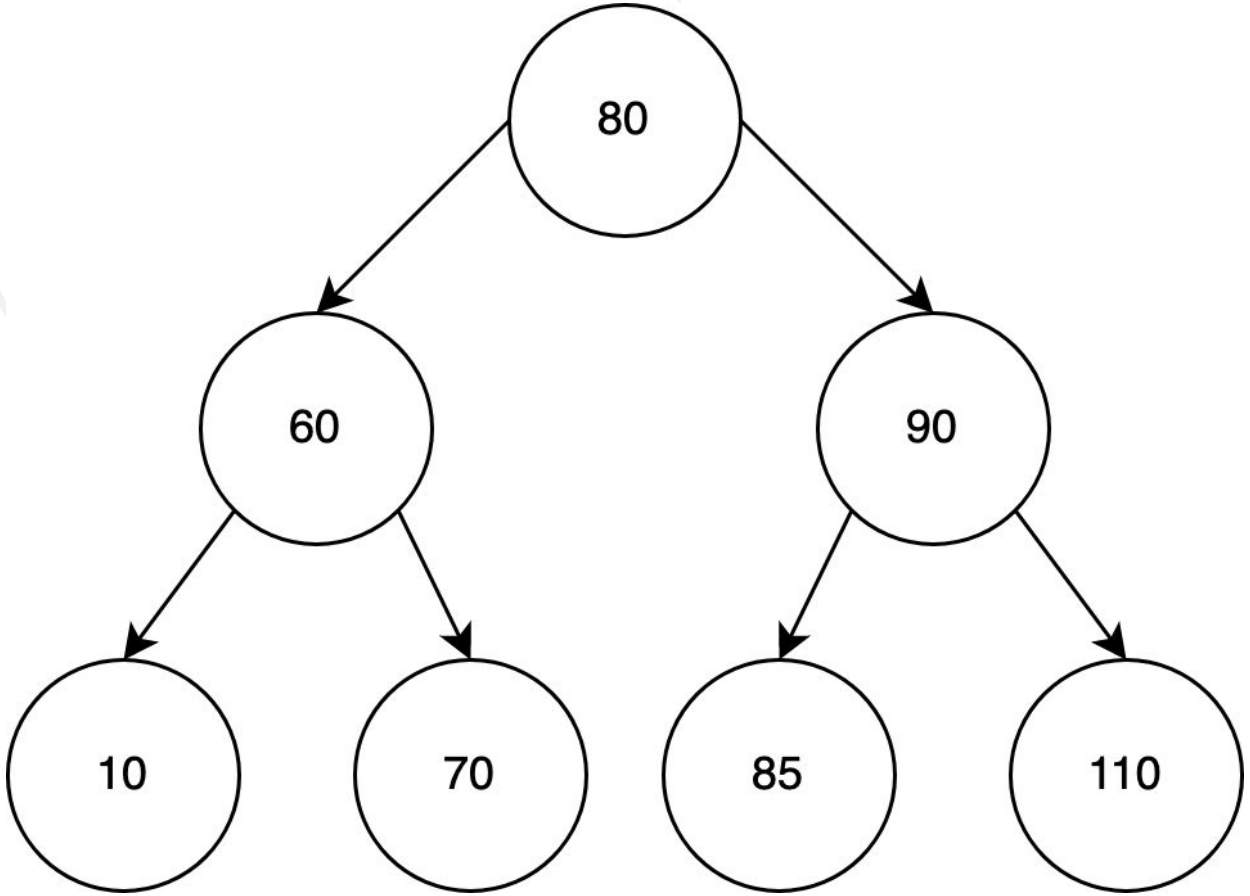
Complexity?
O(log n)



BSTs: Search Implementation



BSTs: Search Implementation



search(Node(80), 85)

search(Node(90), 85)

search(Node(85), 85)

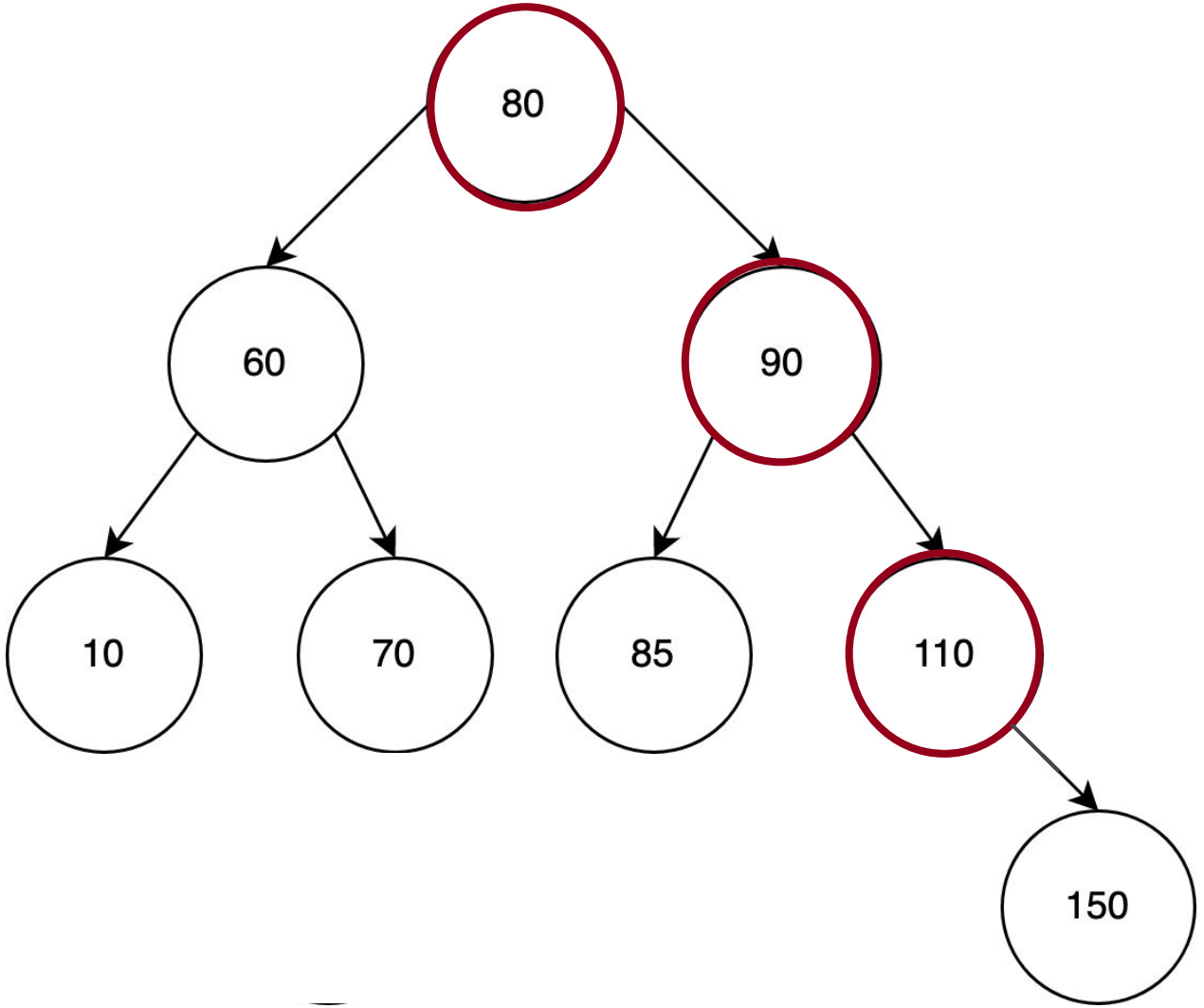
Today's Lecture

1. Binary Search Trees
2. Search
3. **Insertion**
4. Removal
5. Summary

Binary Search Trees: Insertion

Insertion must maintain the properties of a BST!

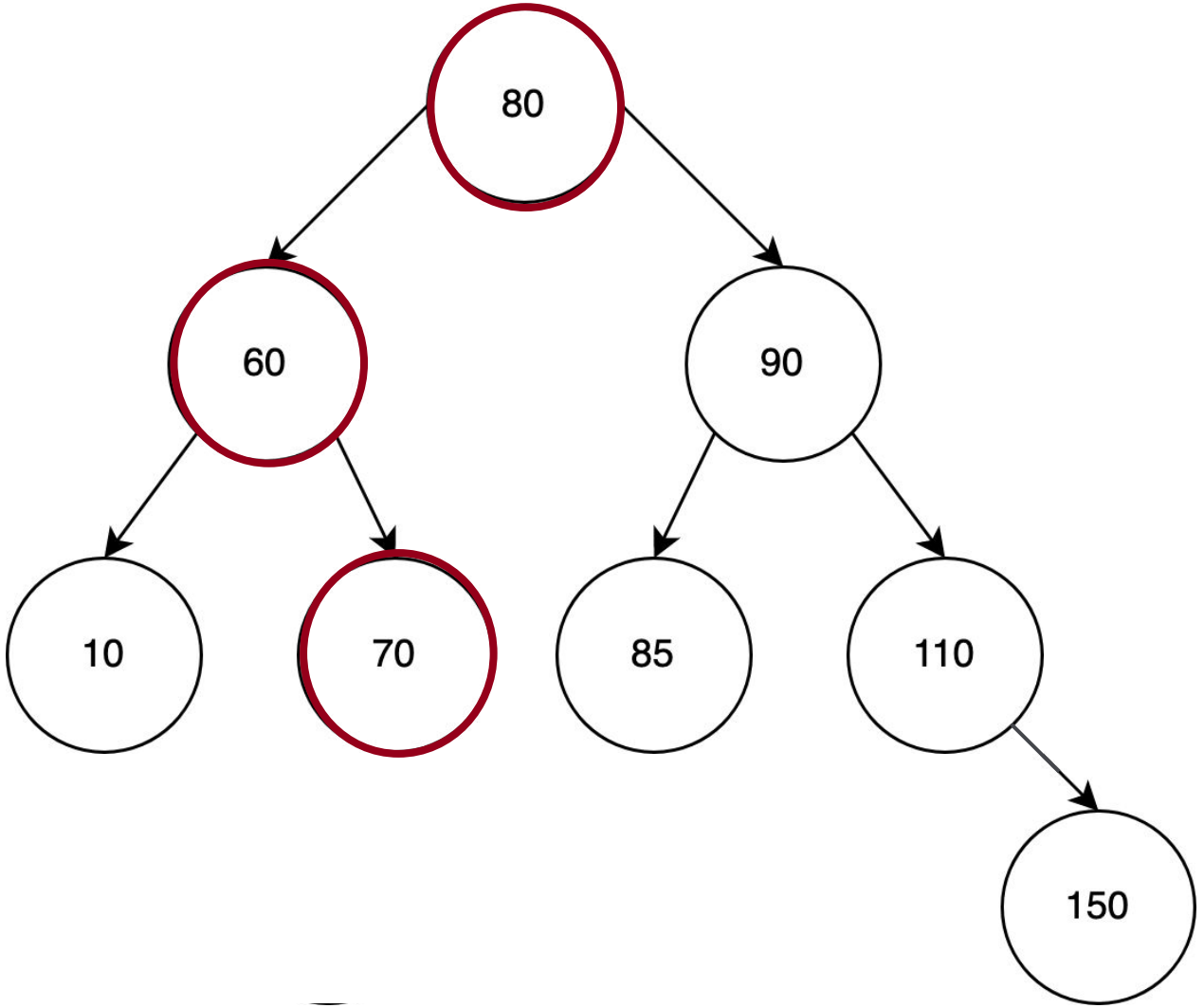
Insert: 150



Binary Search Trees: Insertion

Insertion must maintain the properties of a BST!

Insert: 64



Complexity?
O(log n)

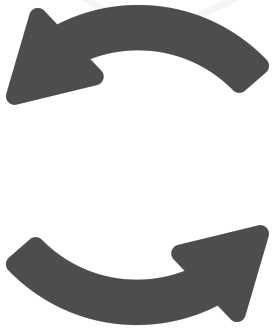
Today's Lecture

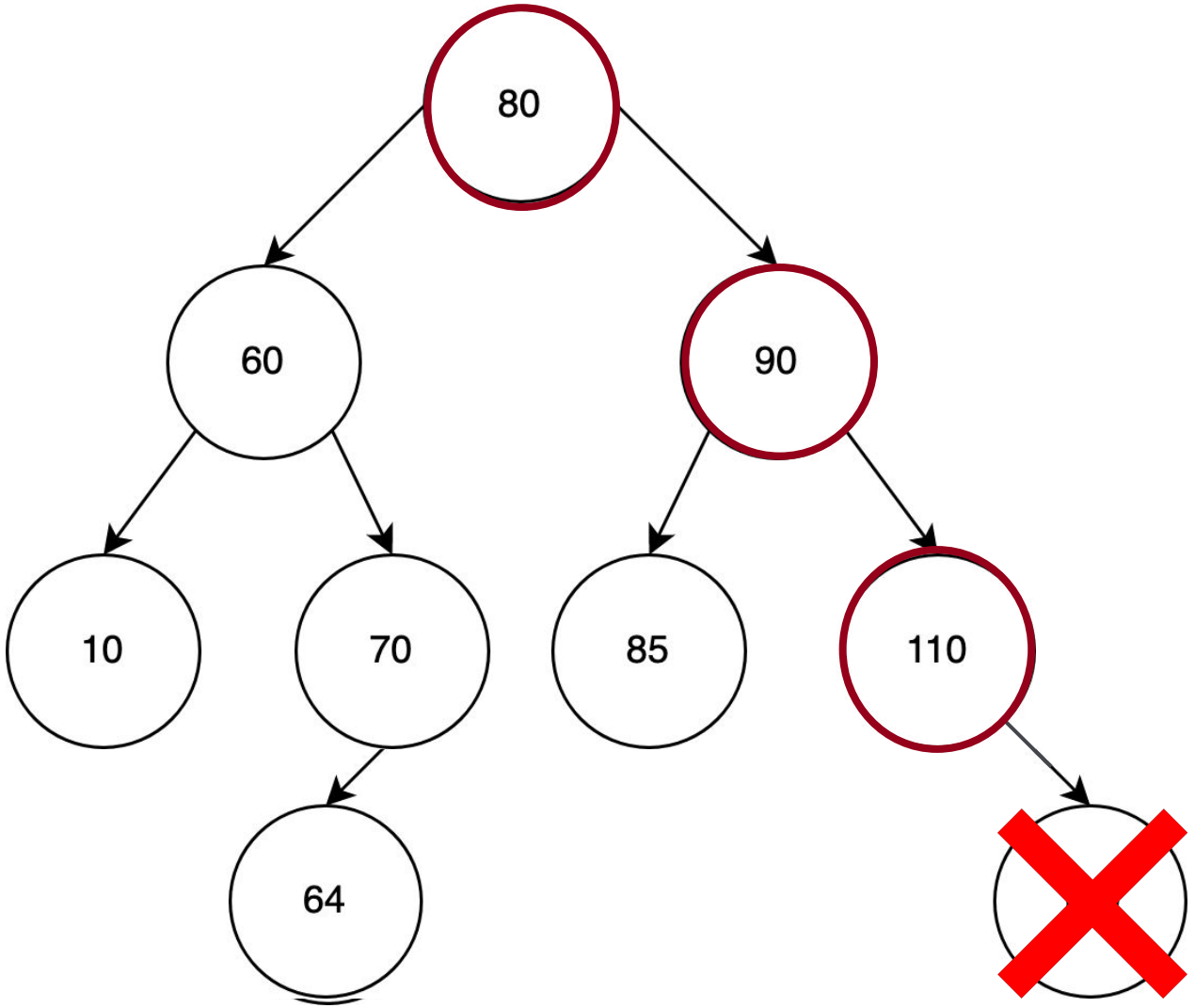
1. Binary Search Trees
2. Search
3. Insertion
4. **Removal**
5. Summary

Binary Search Trees: Deletion

Deletion must maintain the properties of a BST!

Delete: 150

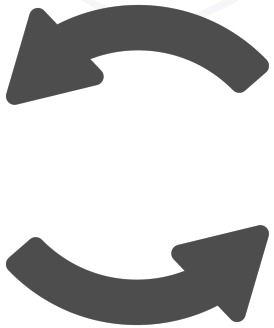


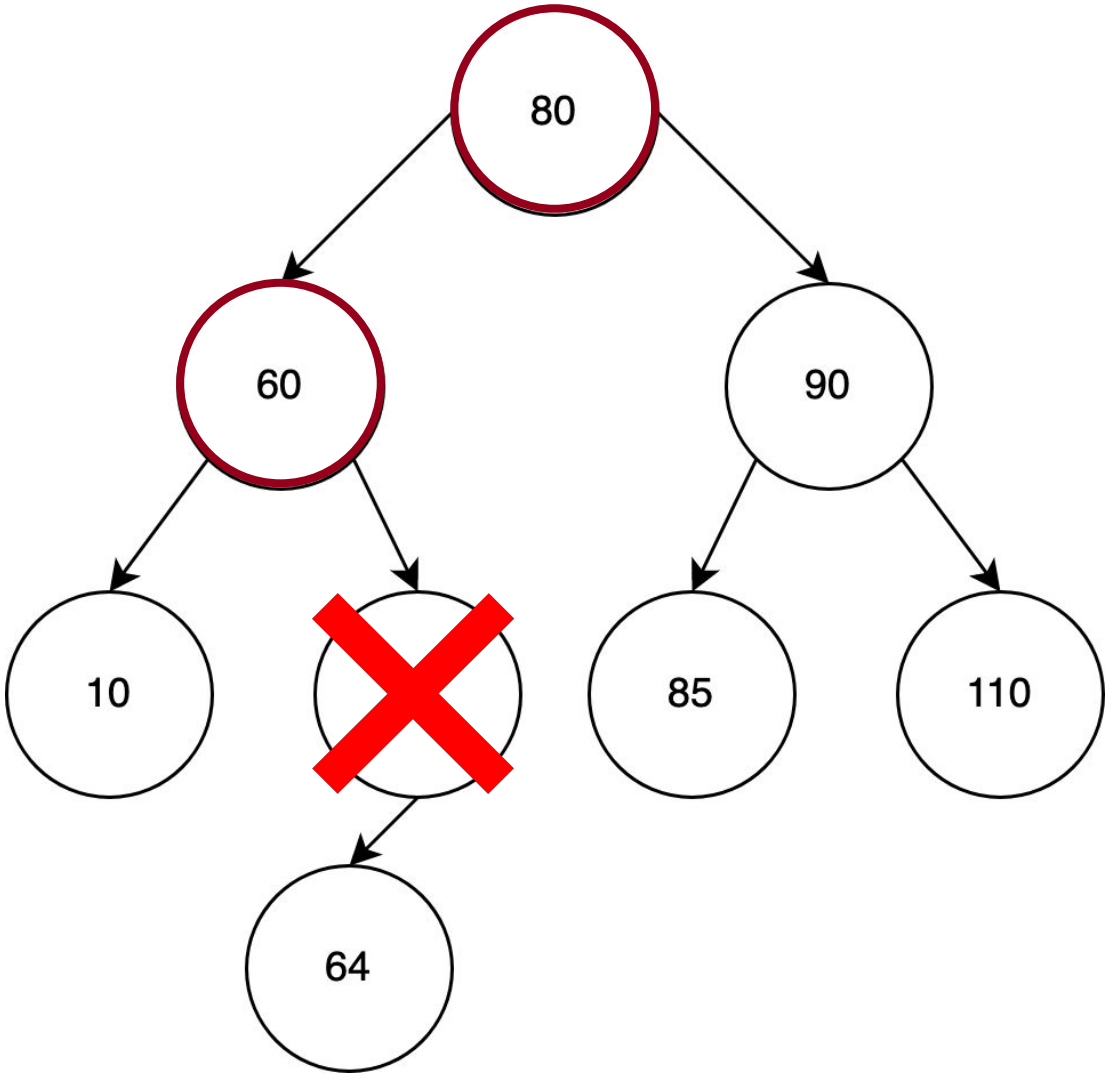


Binary Search Trees: Deletion

Deletion must maintain the properties of a BST!

Delete: 70





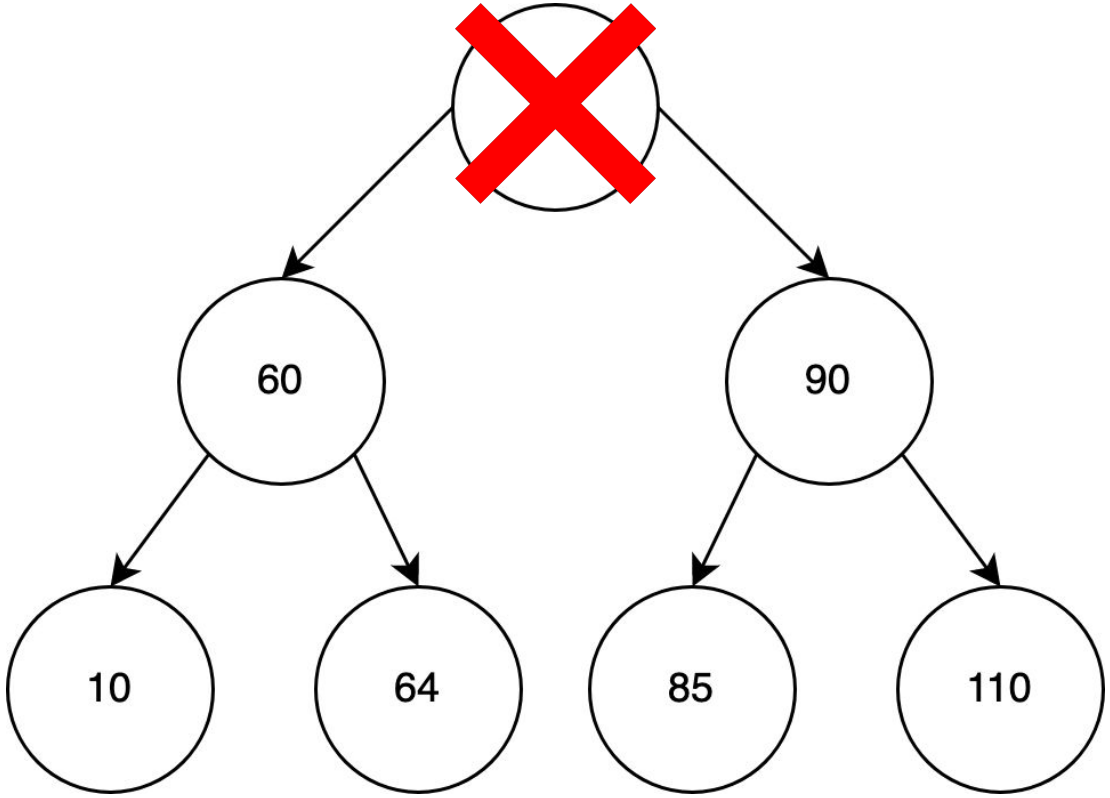
Binary Search Trees: Deletion

Deletion must maintain the properties of a BST!

Delete: 80

At **each node** with value **k**

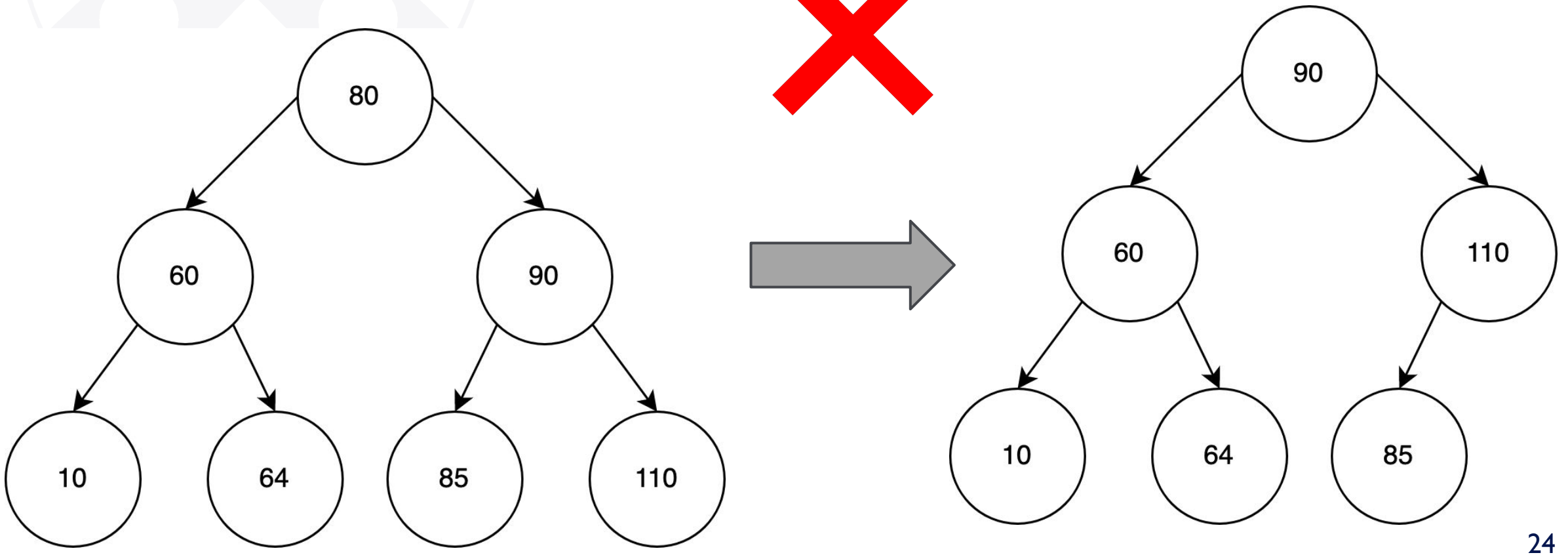
- Left subtree contains only nodes with value **lesser** than **k**
- Right subtree contains only nodes with value **greater** than **k**
- Both subtrees are a **binary search tree**



Binary Search Trees: Deletion

Replace with 90?

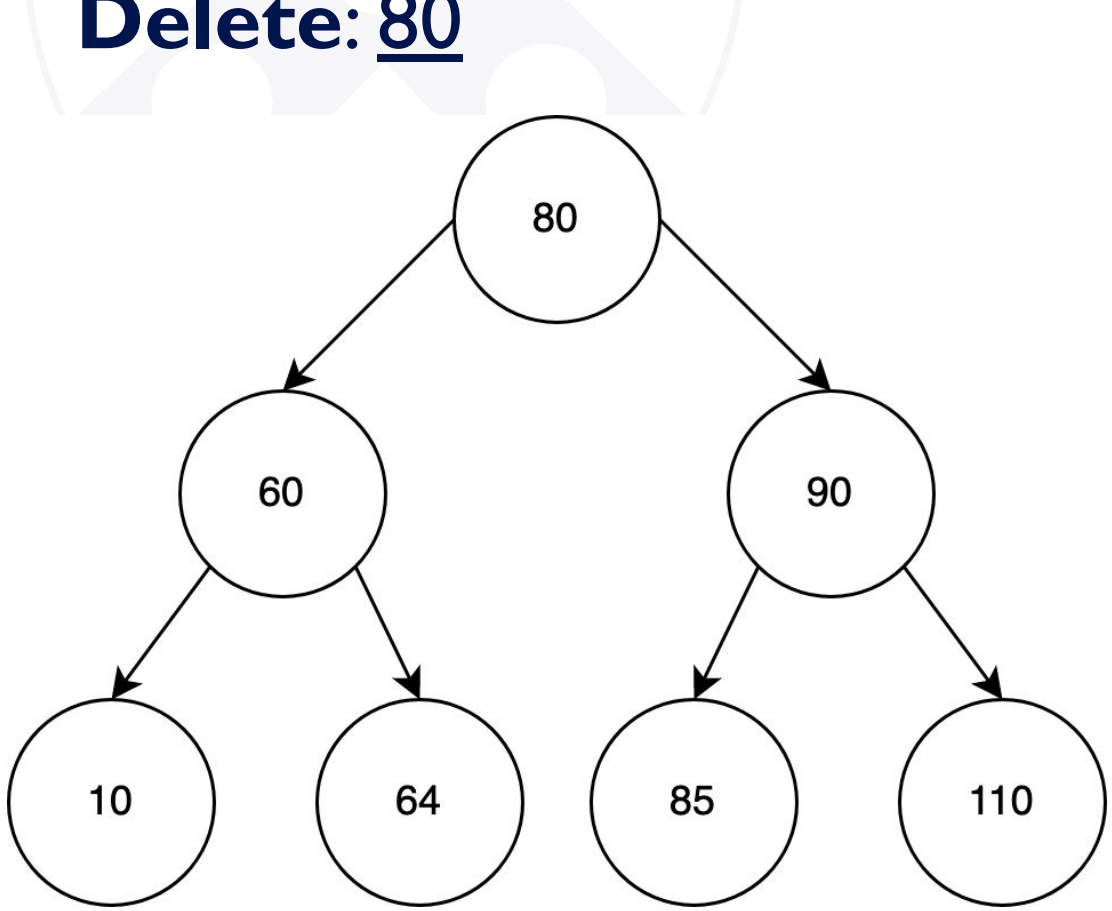
Delete: 80

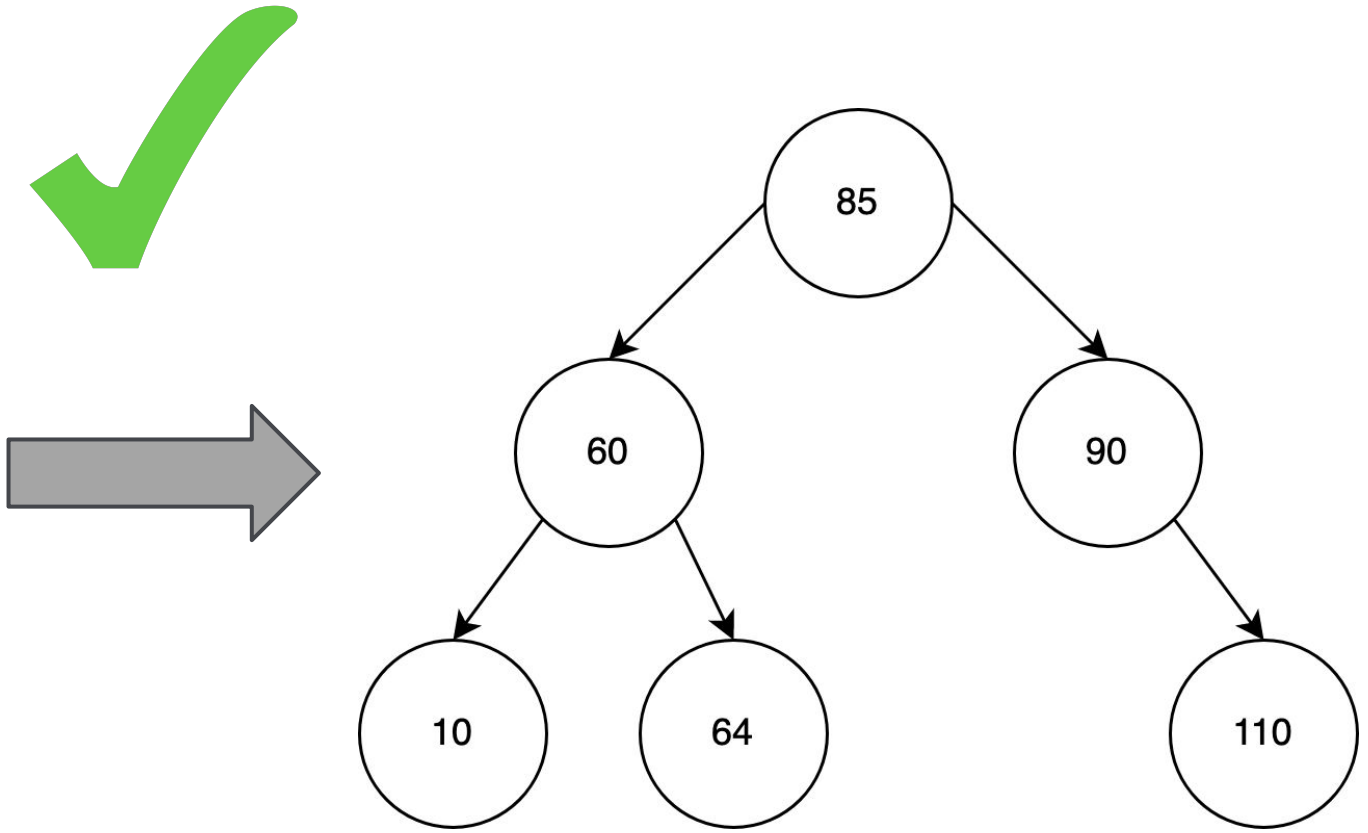


Binary Search Trees: Deletion

Replace with 85?

Delete: 80

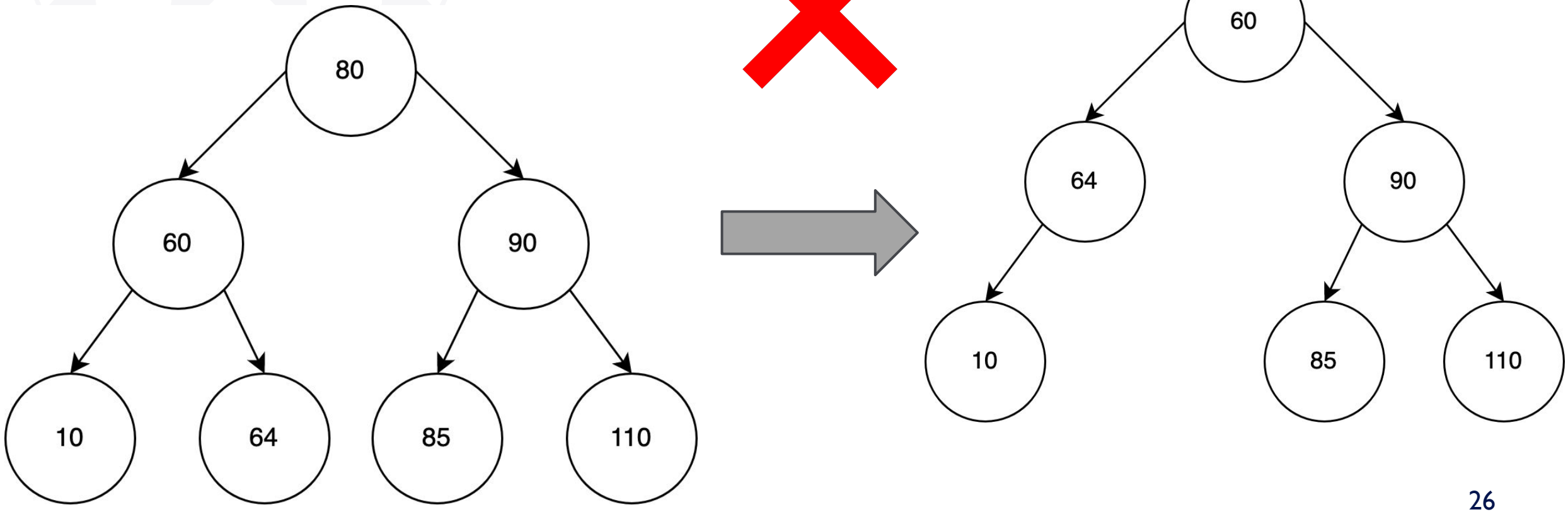




Binary Search Trees: Deletion

Replace with 60?

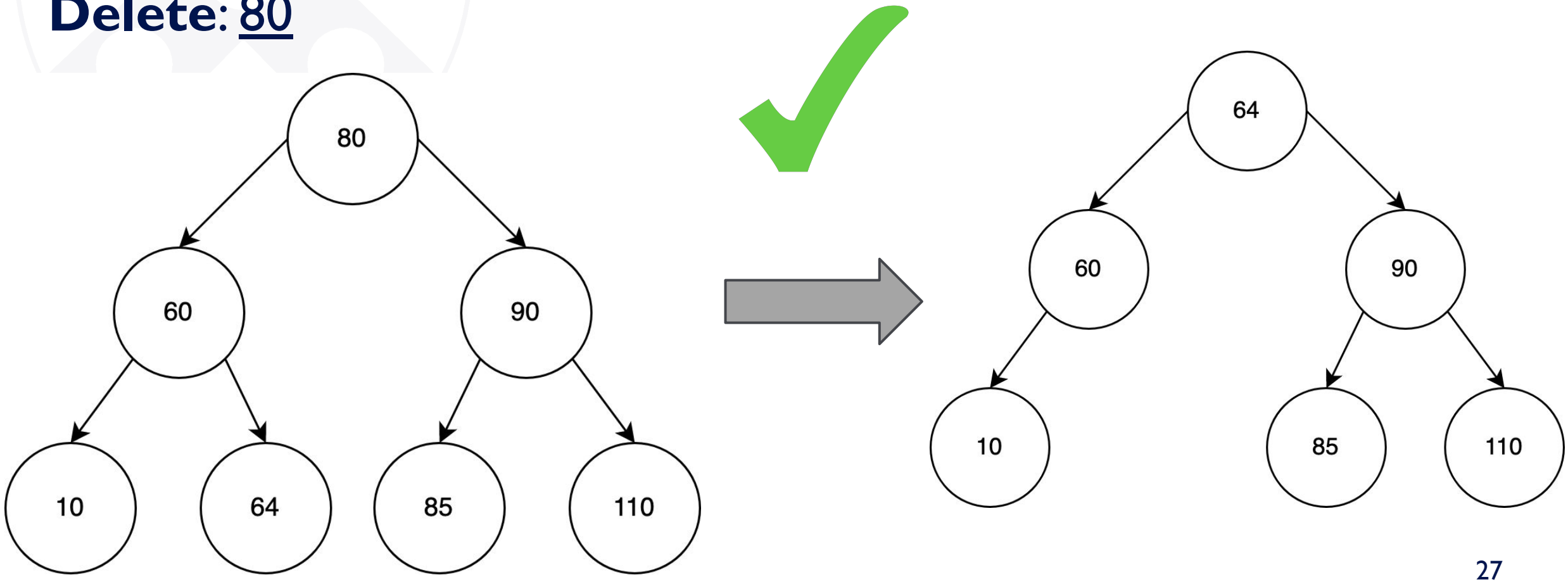
Delete: 80



Binary Search Trees: Deletion

Replace with 64?

Delete: 80



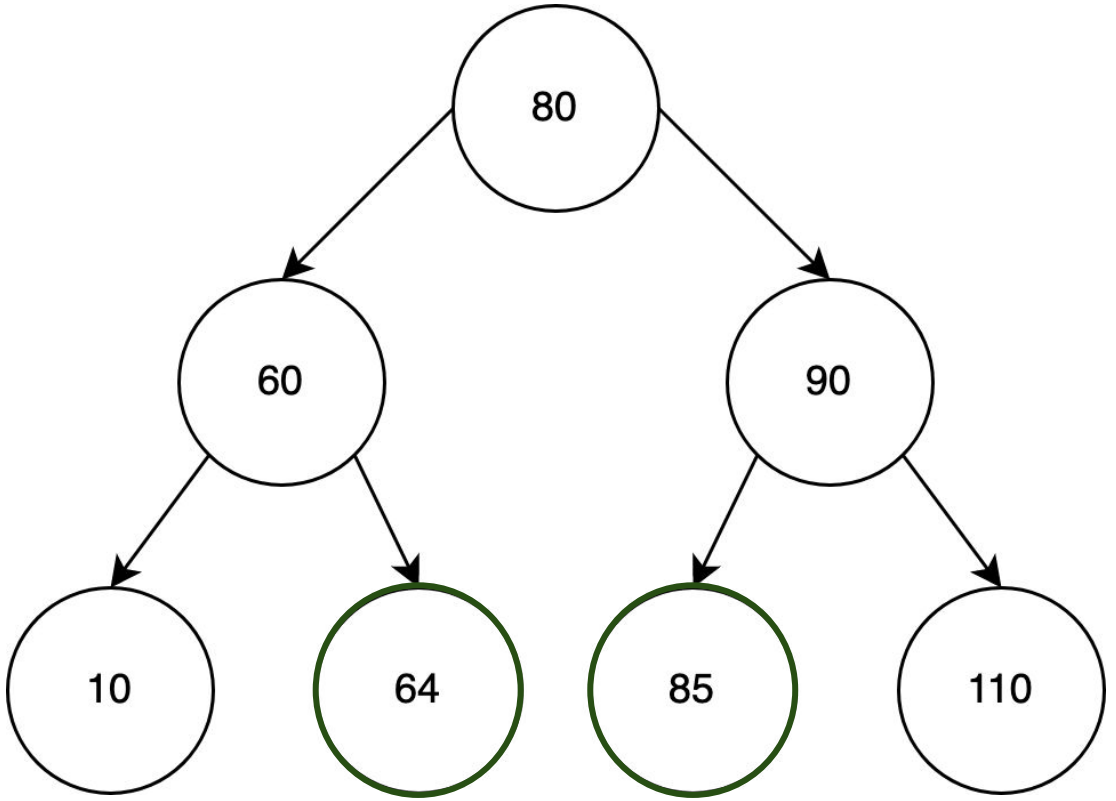
Binary Search Trees: Deletion

Deletion must maintain the properties of a BST!

Delete: 80

Replace deleted node with either:

1. Smallest value in right subtree
2. Largest value in left subtree



Binary Search Trees: Deletion

Complexity?

Case 1: Removing a **leaf node**

O(log n)

Case 2: Removing a **node with one child**

O(log n)

Case 3: Removing a **node with two children**

O(log n)

What can go wrong?

Complexity?

Search

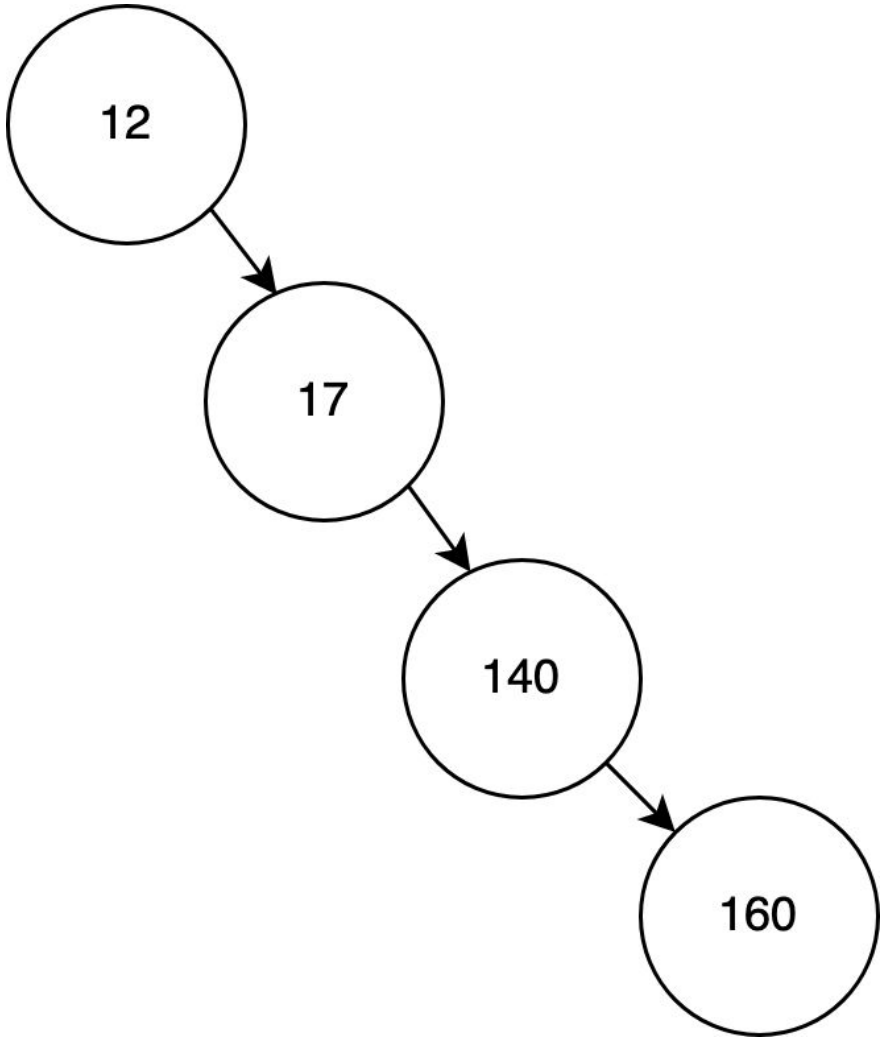
O(n)

Insertion:

O(n)

Deletion:

O(n)



Tree Traversals

Binary Tree Traversals

Traversal visits all nodes in a tree in some order

Inorder:

left subtree, current, right subtree

will print “in order” (increasing values)

Preorder:

current, left subtree, right subtree

Postorder:

left subtree, right subtree, current

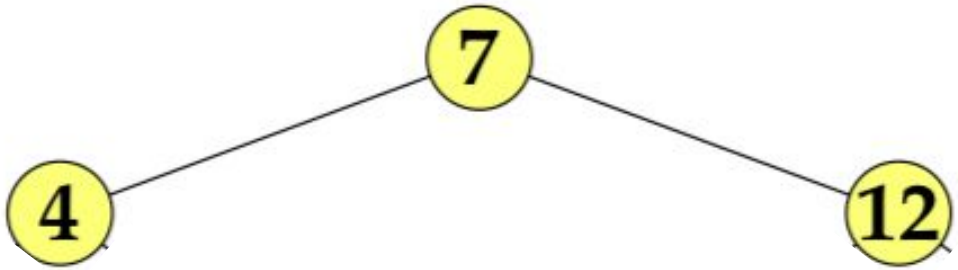
In Order Traversal

1. Move left until you reach a node without a left child
2. Print the current node
3. Move right

Inorder Example 1

What would the in-order traversal be here?

left subtree, current, right subtree



Inorder Example 2

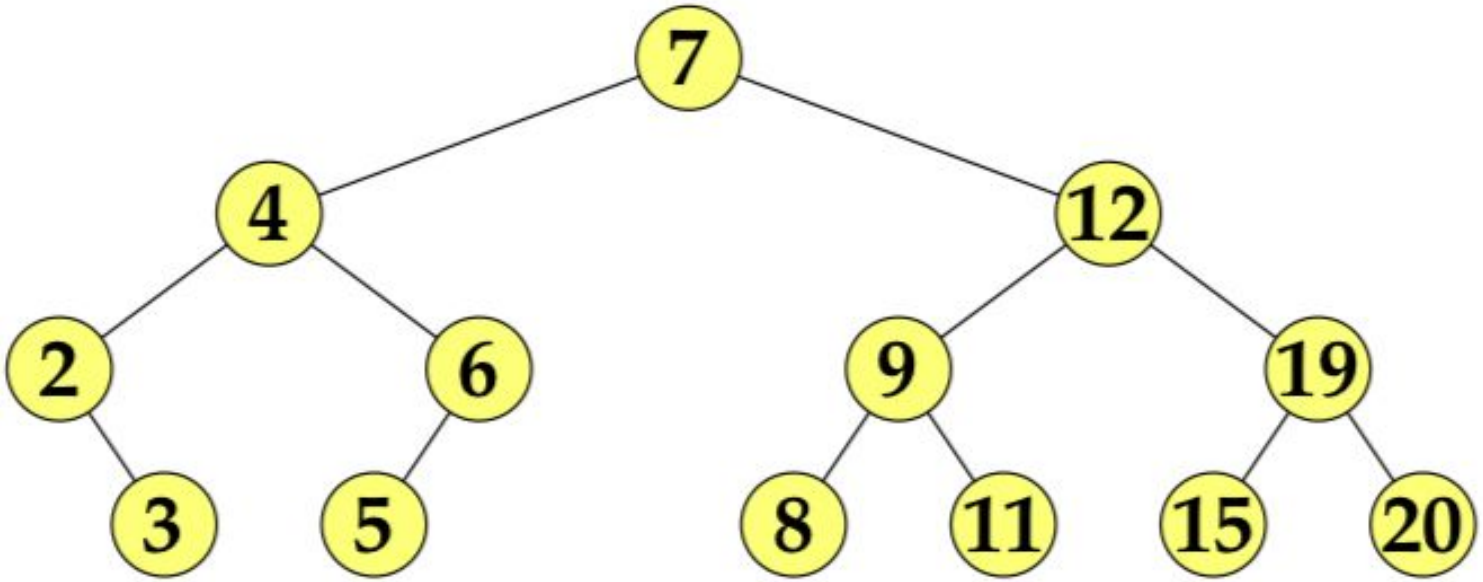
- Larger tree (Height > 1)

- **Process entire left subtree first**
  - **bottom most left node**
  - **current**
  - **bottom most right node**

Inorder Example 2

What would the in-order traversal be here?

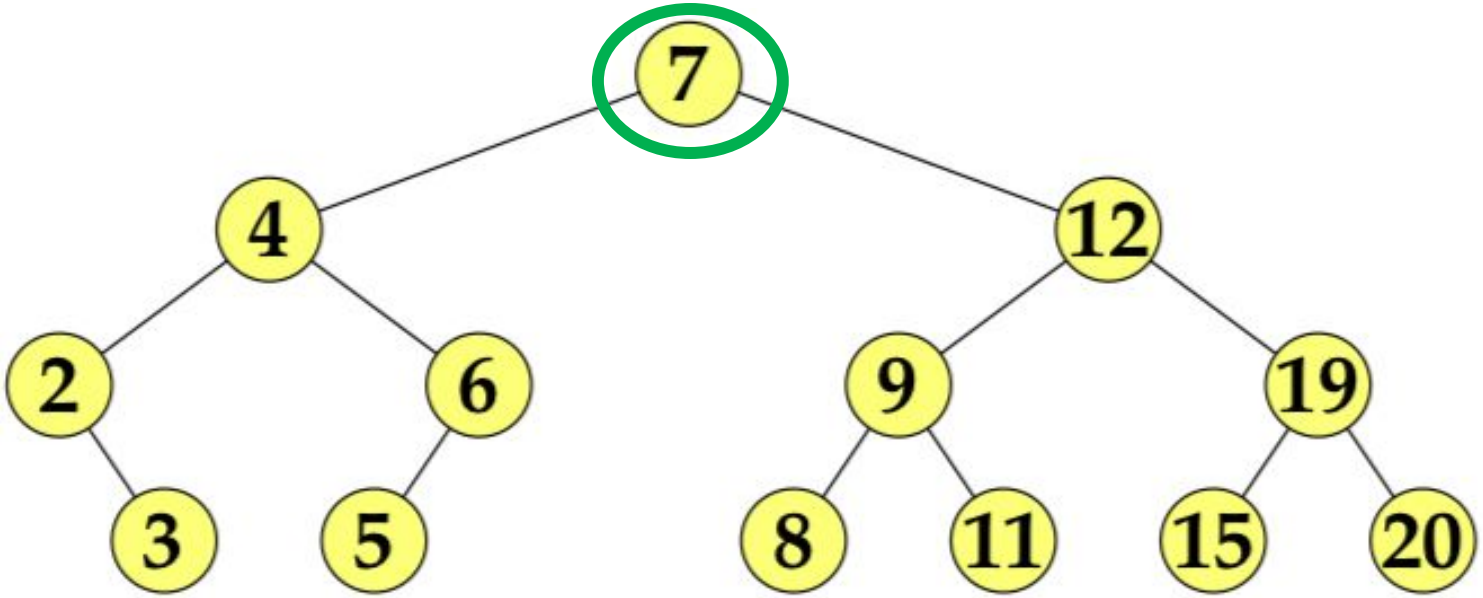
left subtree, current, right subtree



Inorder

What would the in-order traversal be here?

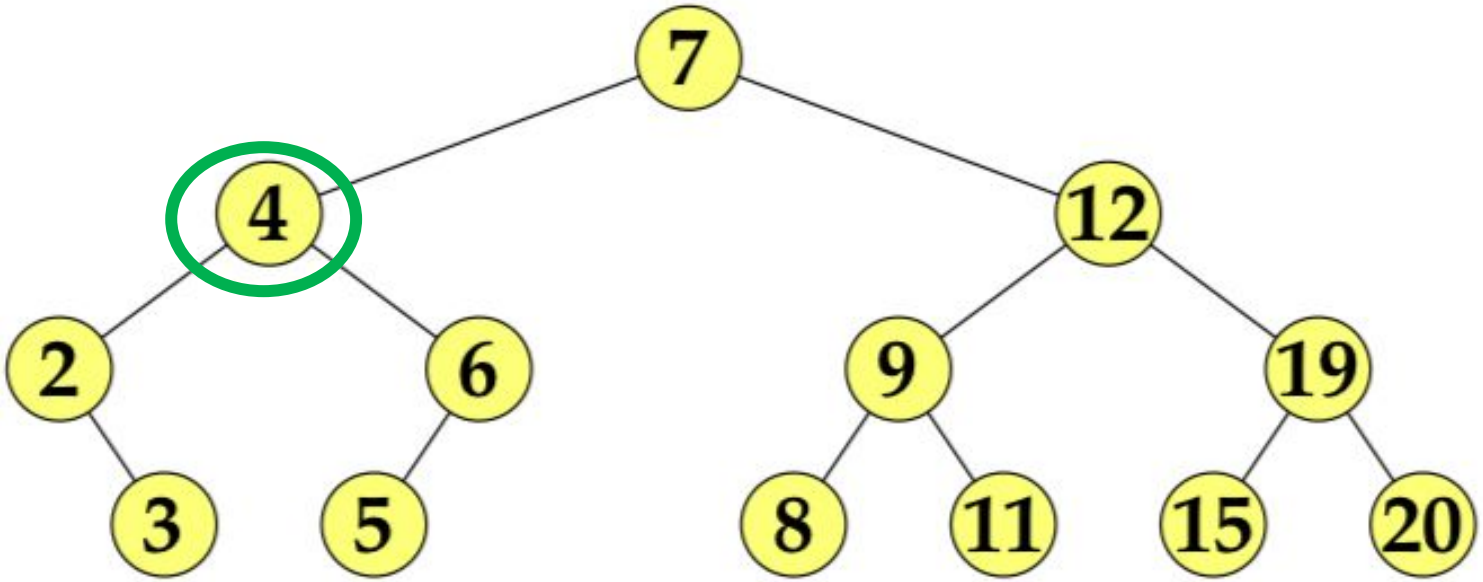
left subtree, current, right subtree



Inorder

What would the in-order traversal be here?

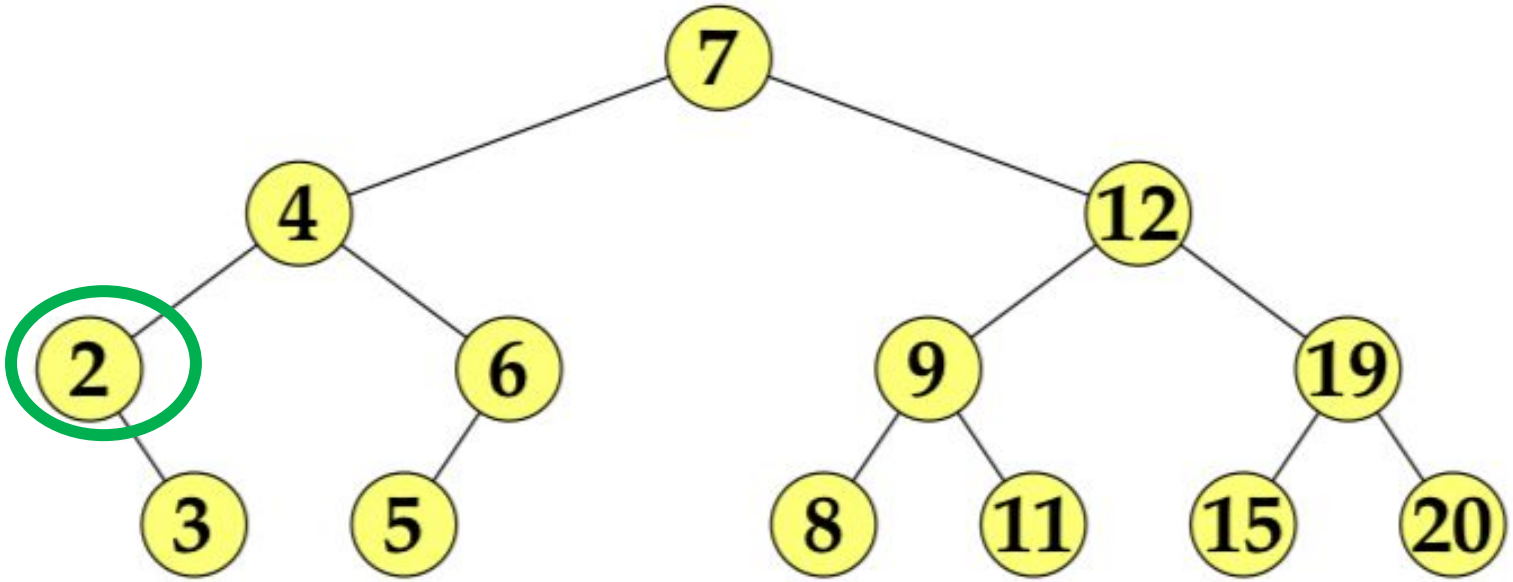
left subtree, current, right subtree



Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

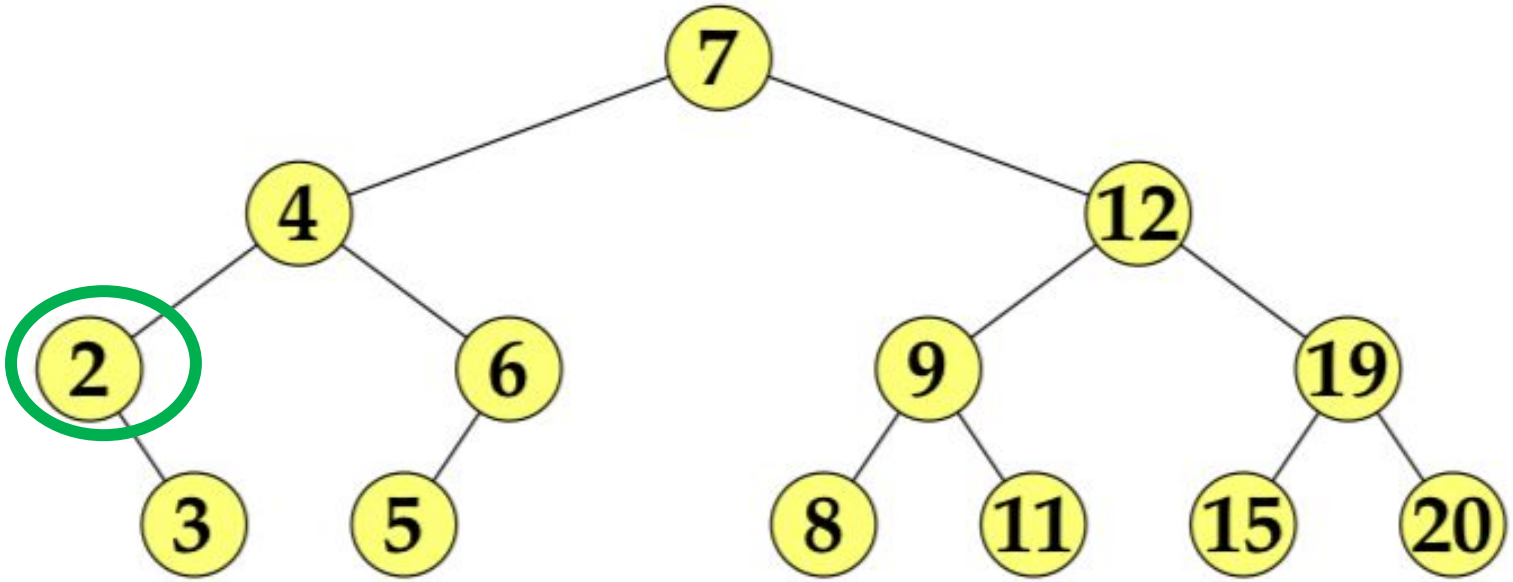


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2,

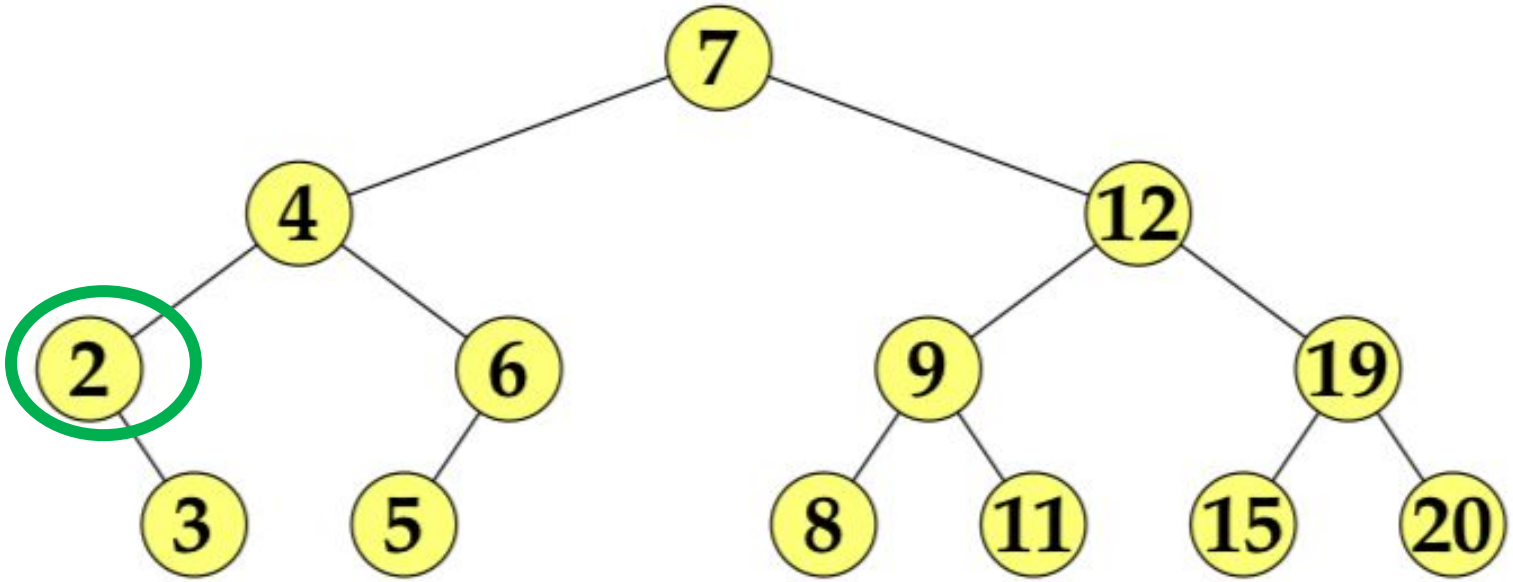


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2,

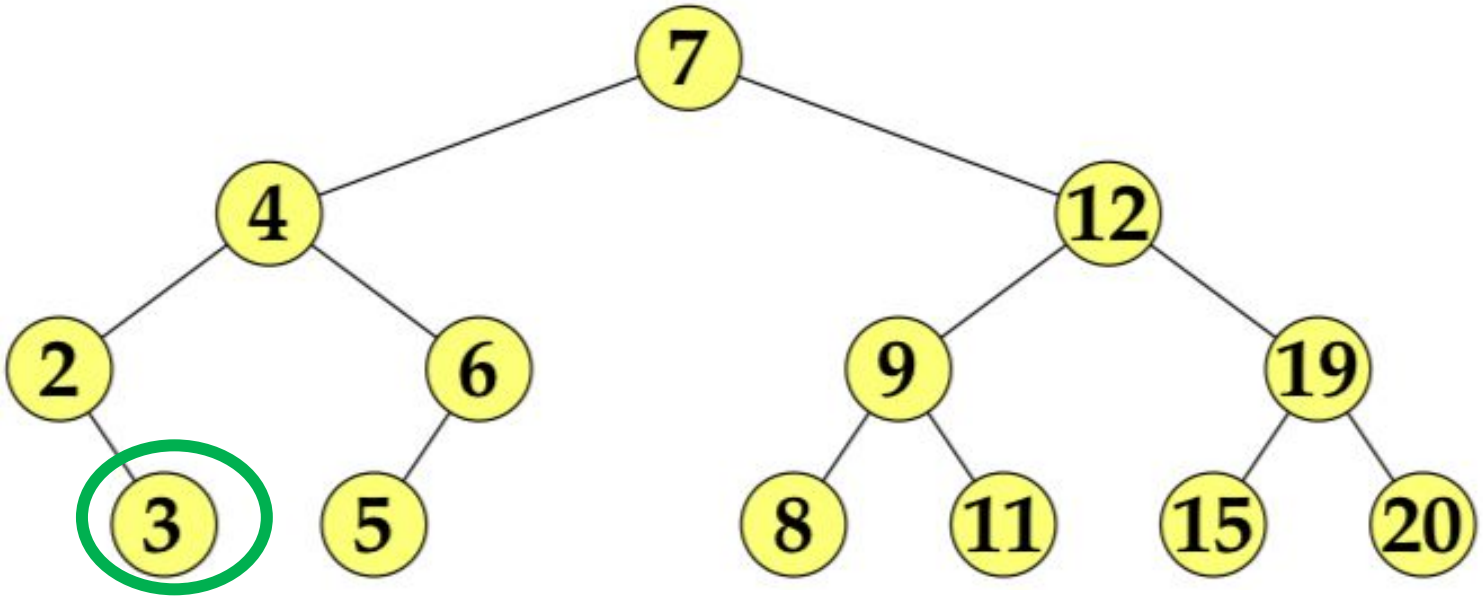


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2,

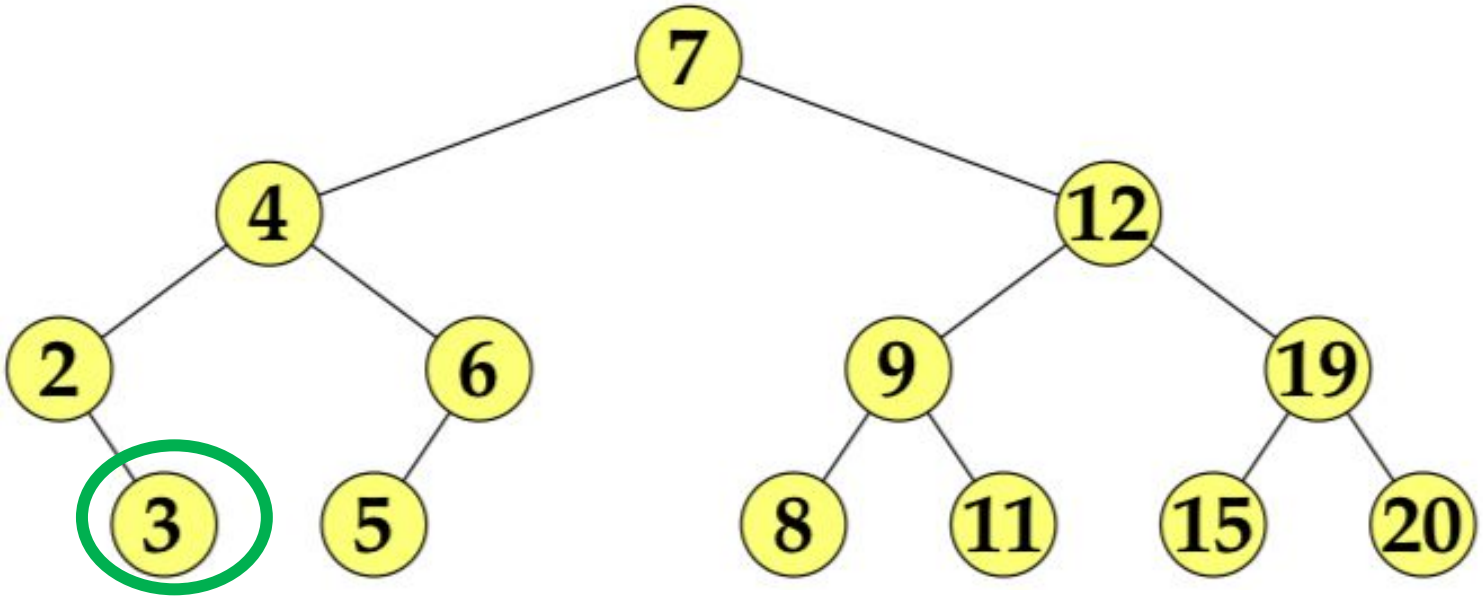


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3

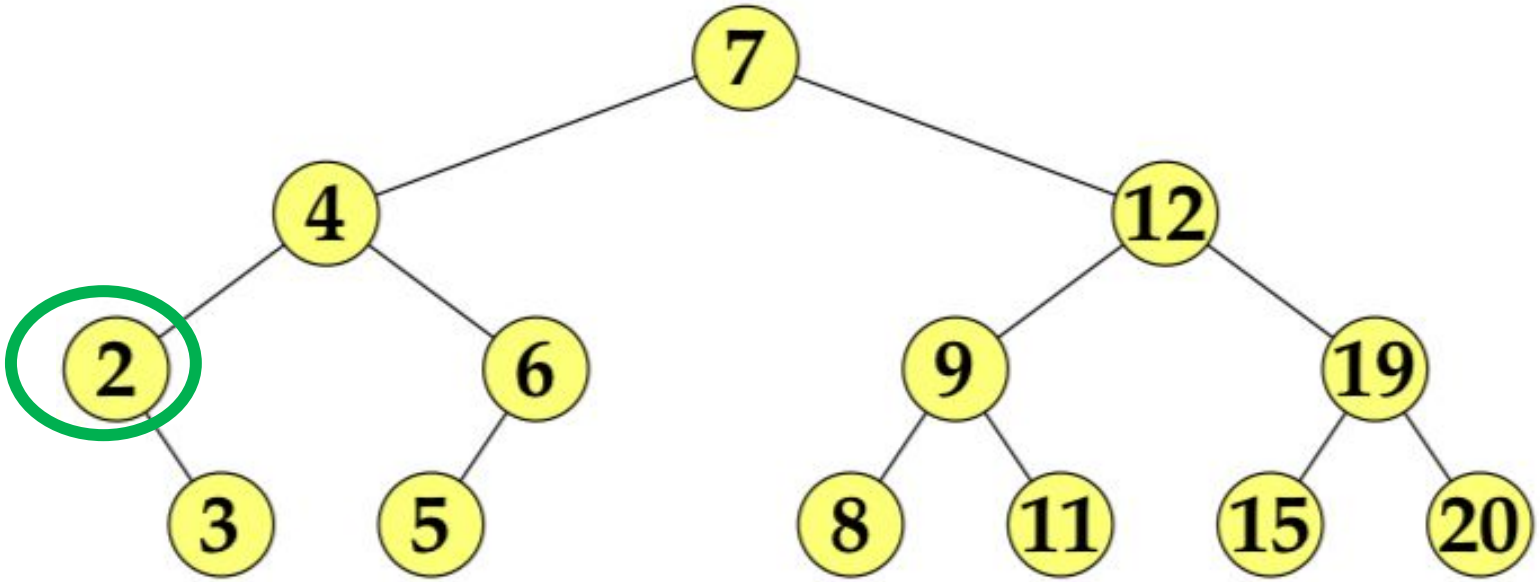


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3

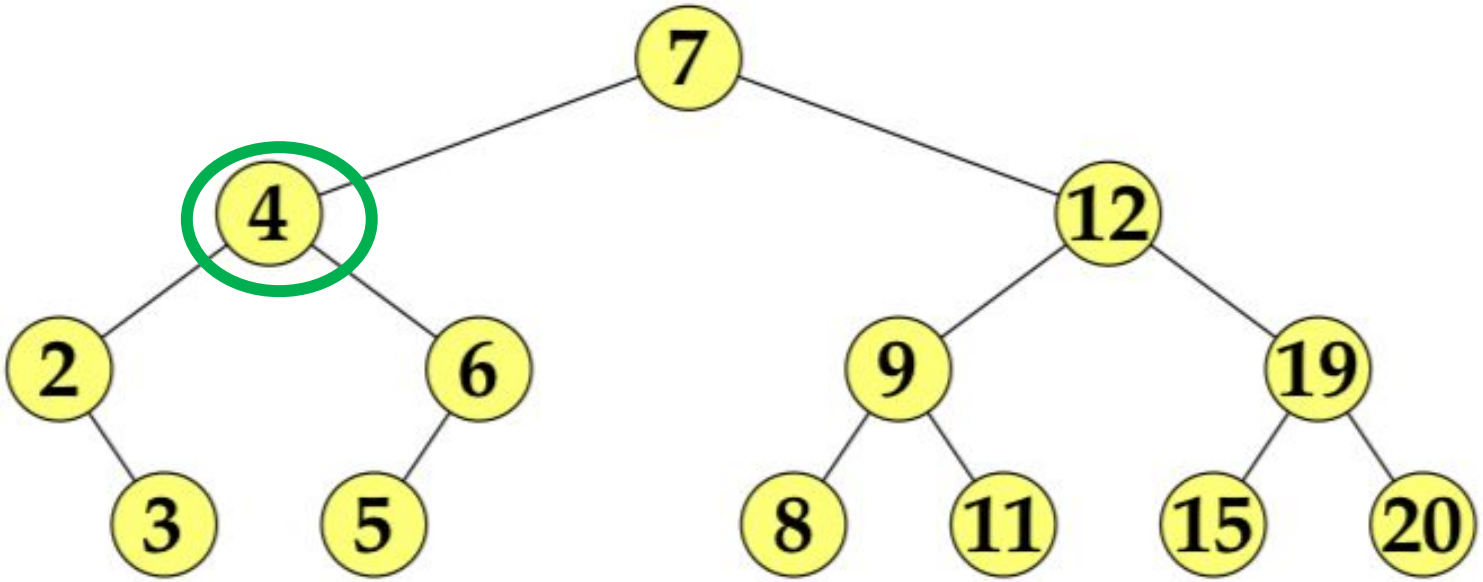


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3

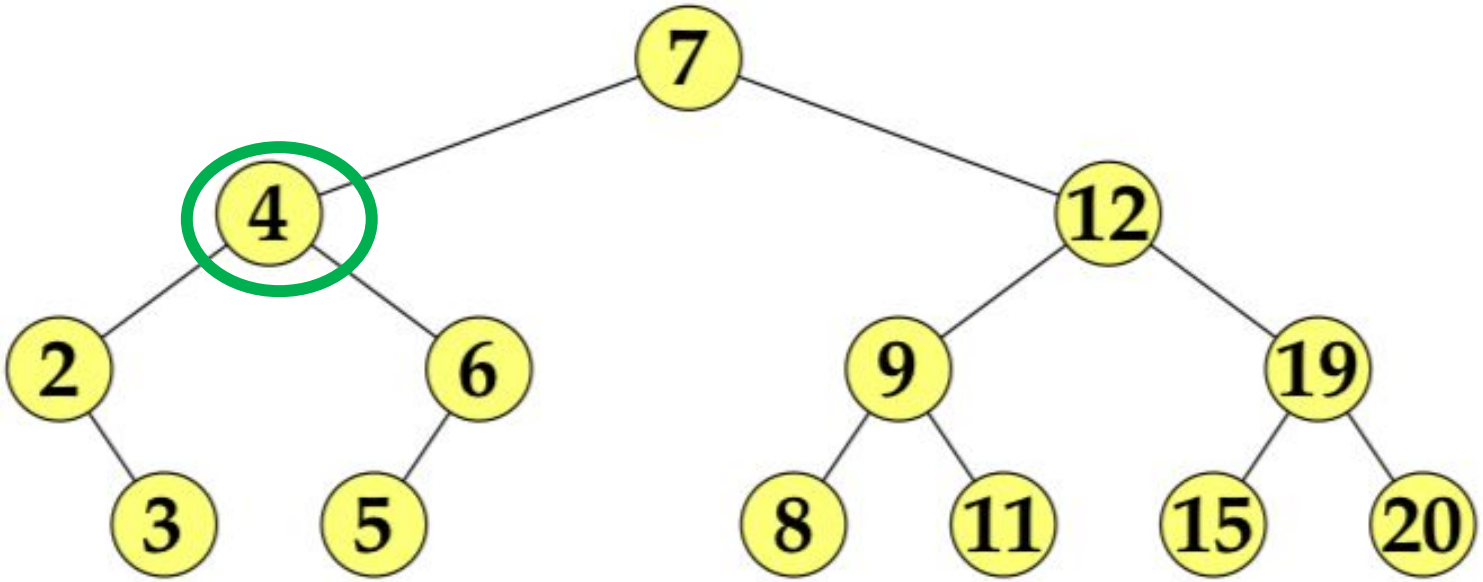


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4

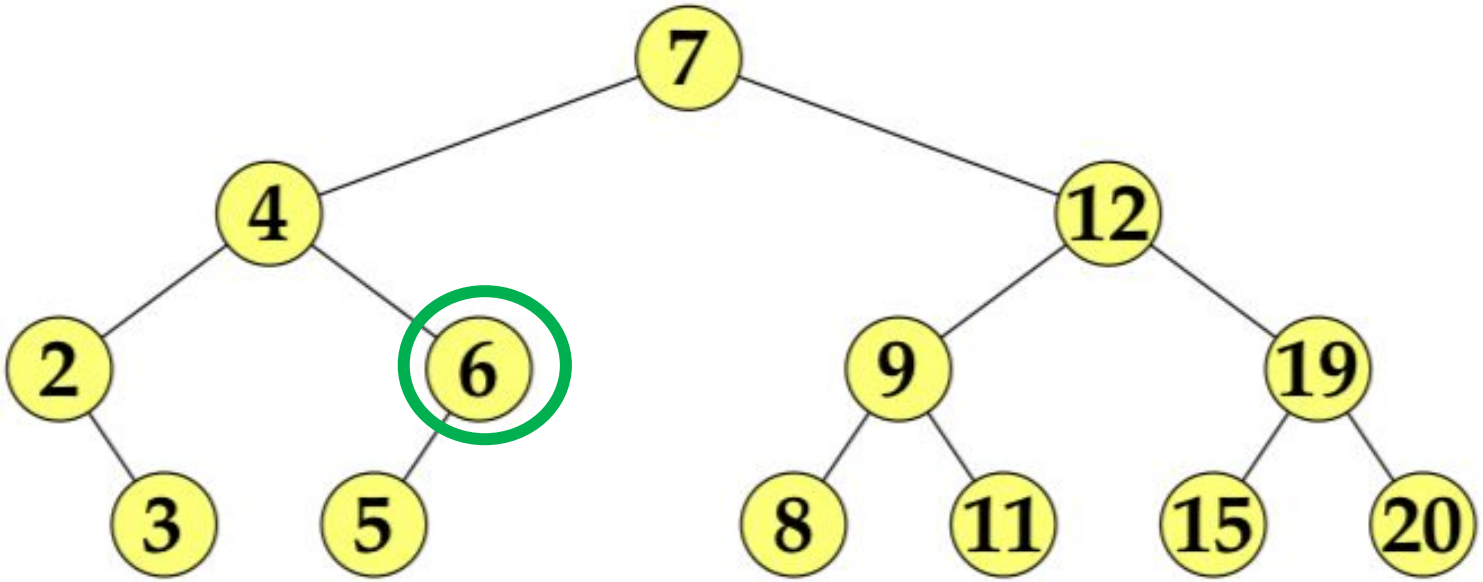


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4

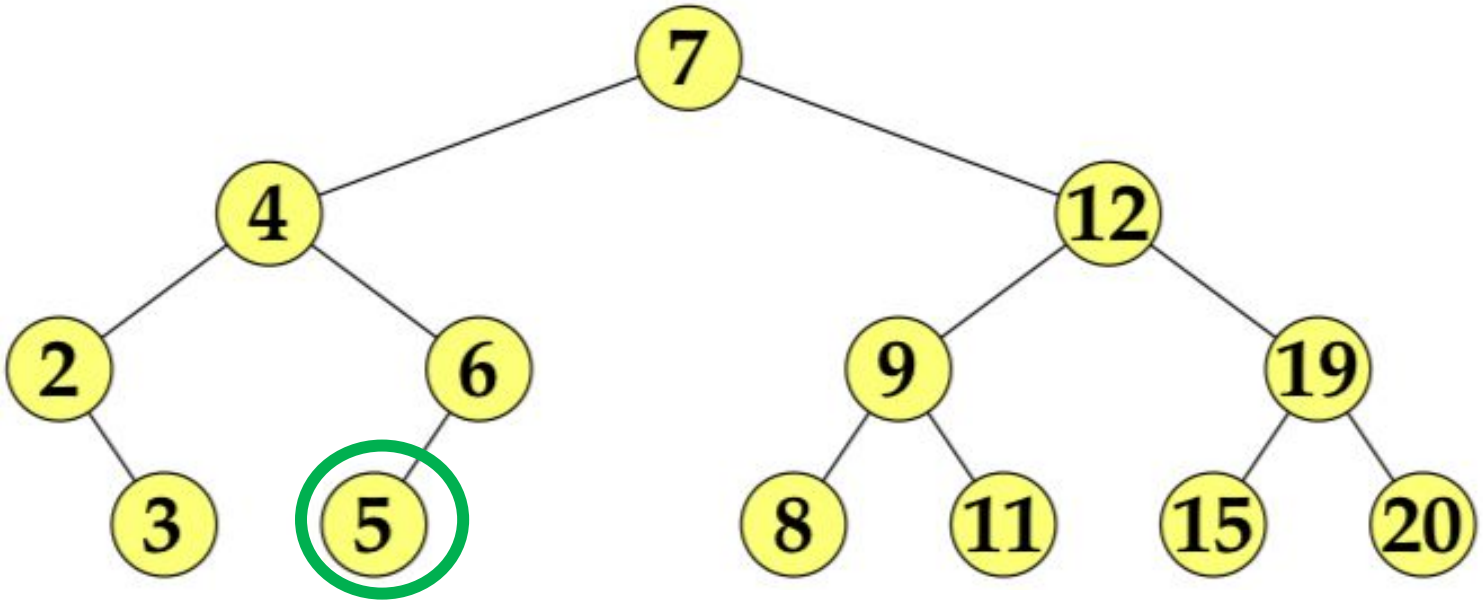


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4

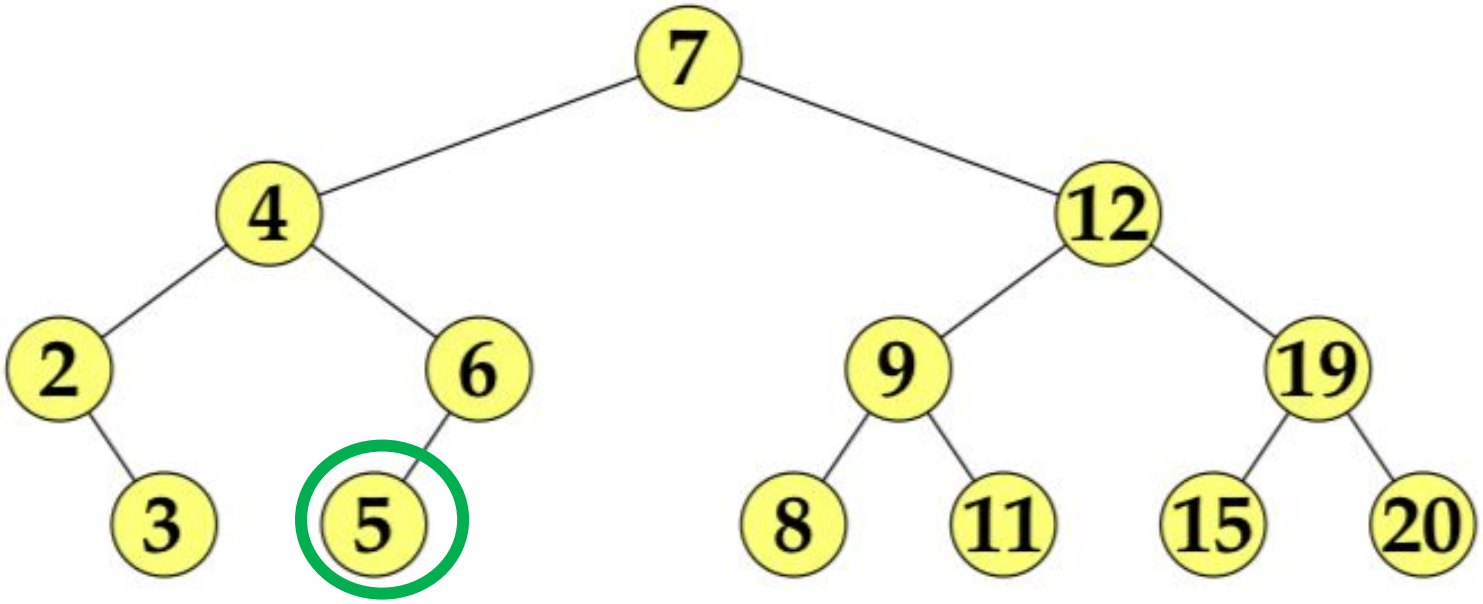


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5

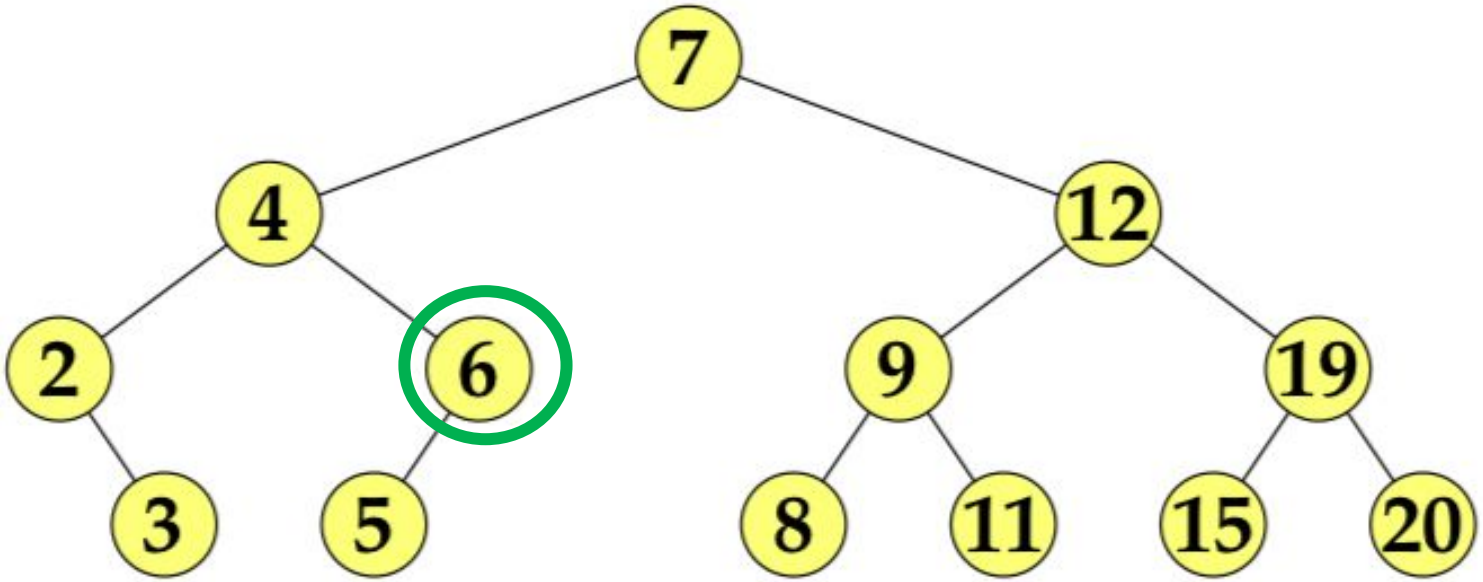


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5

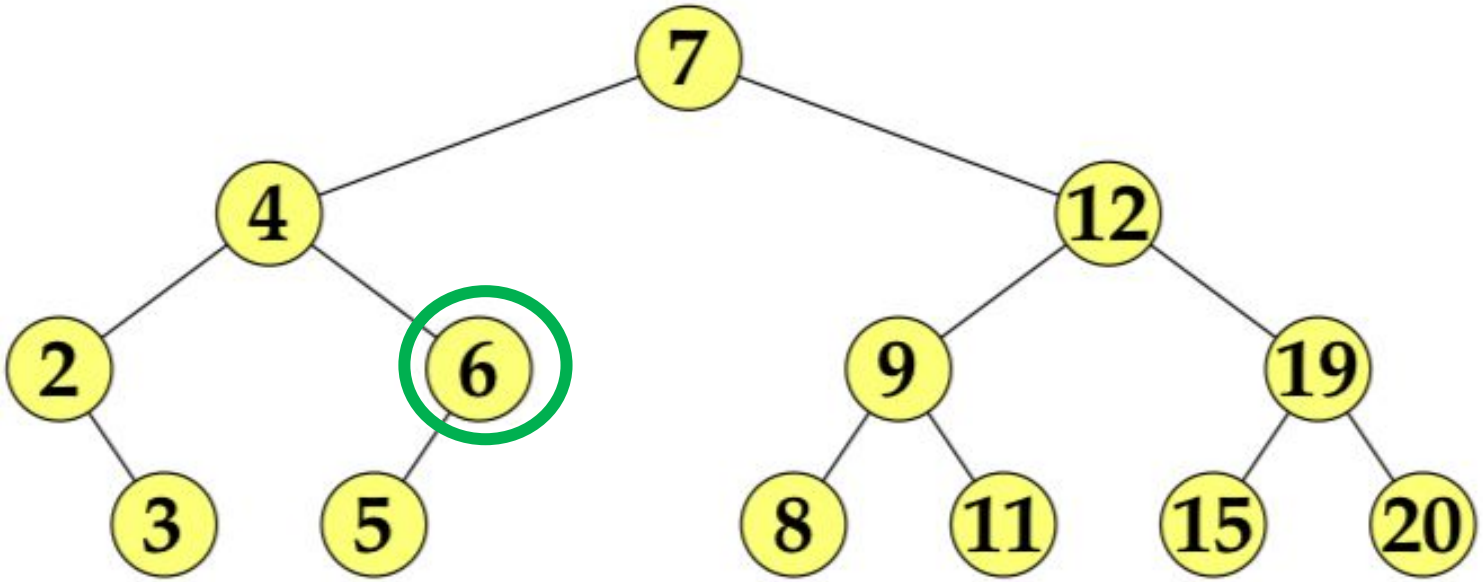


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6

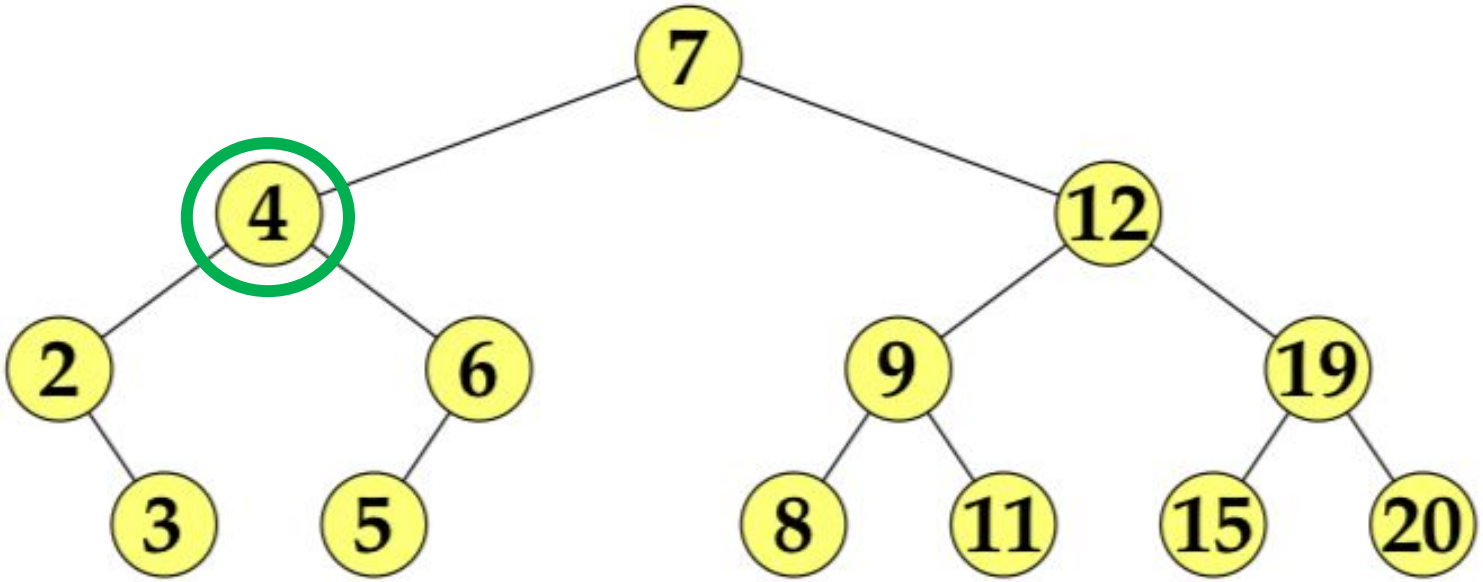


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6

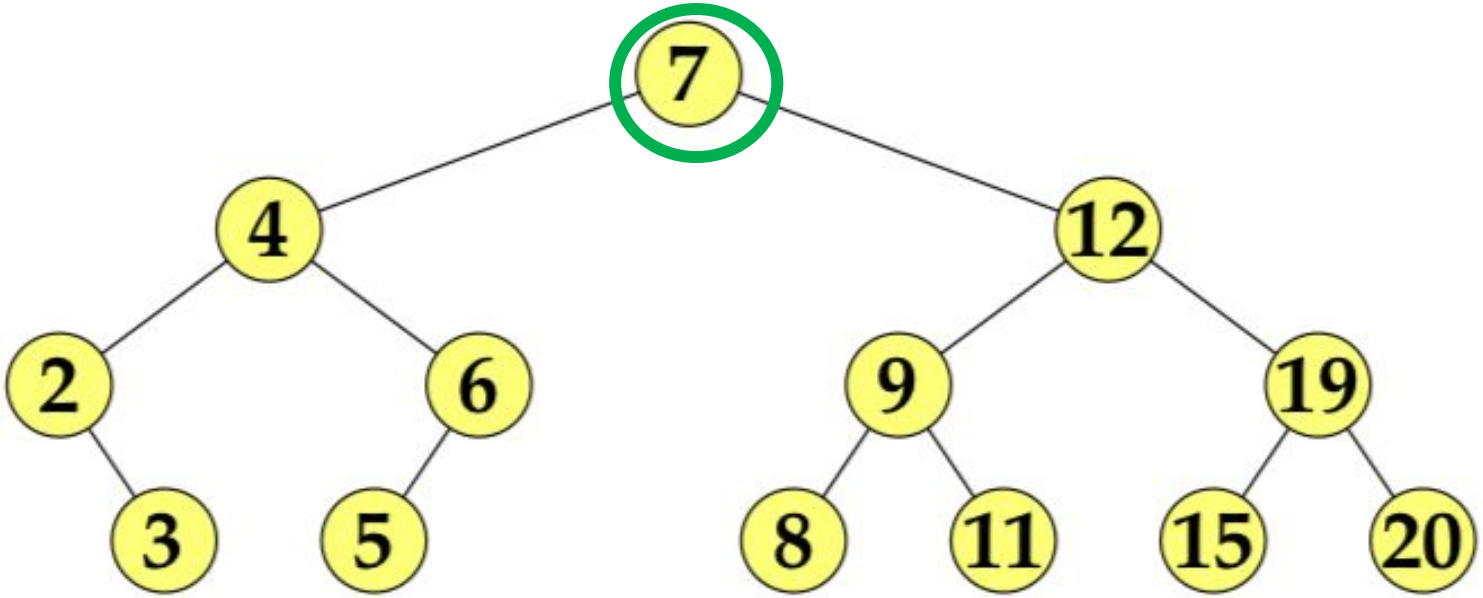


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6

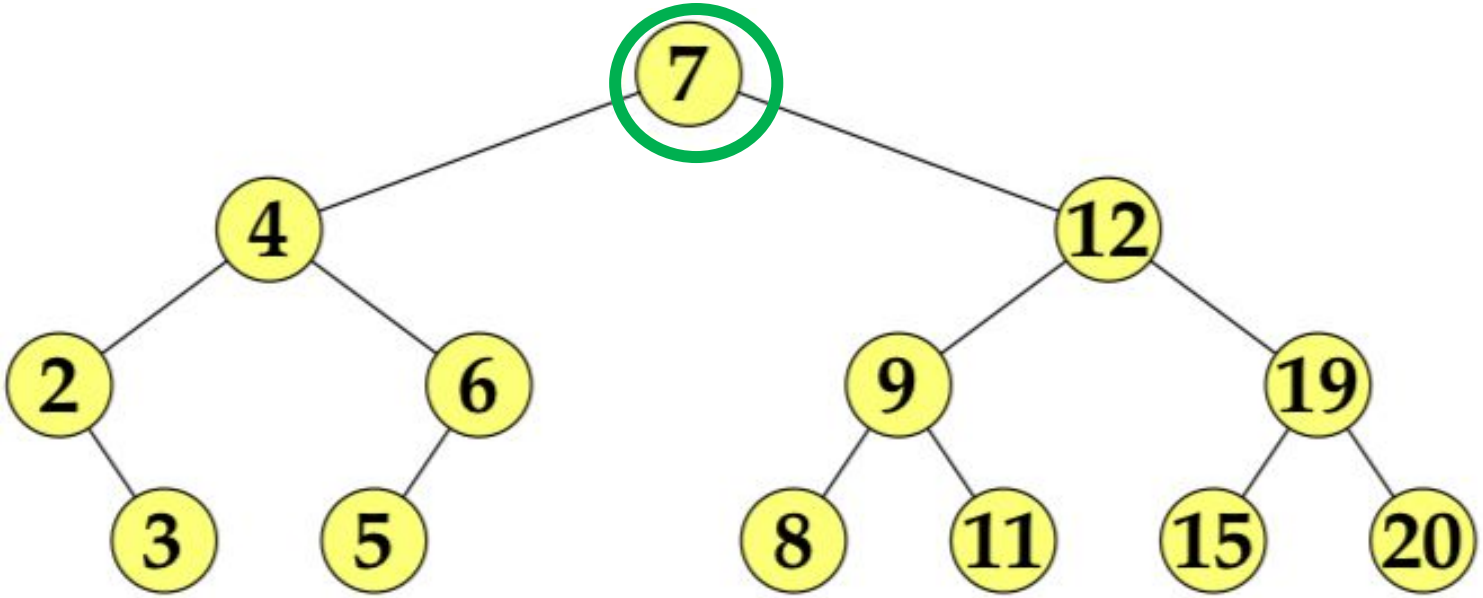


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7

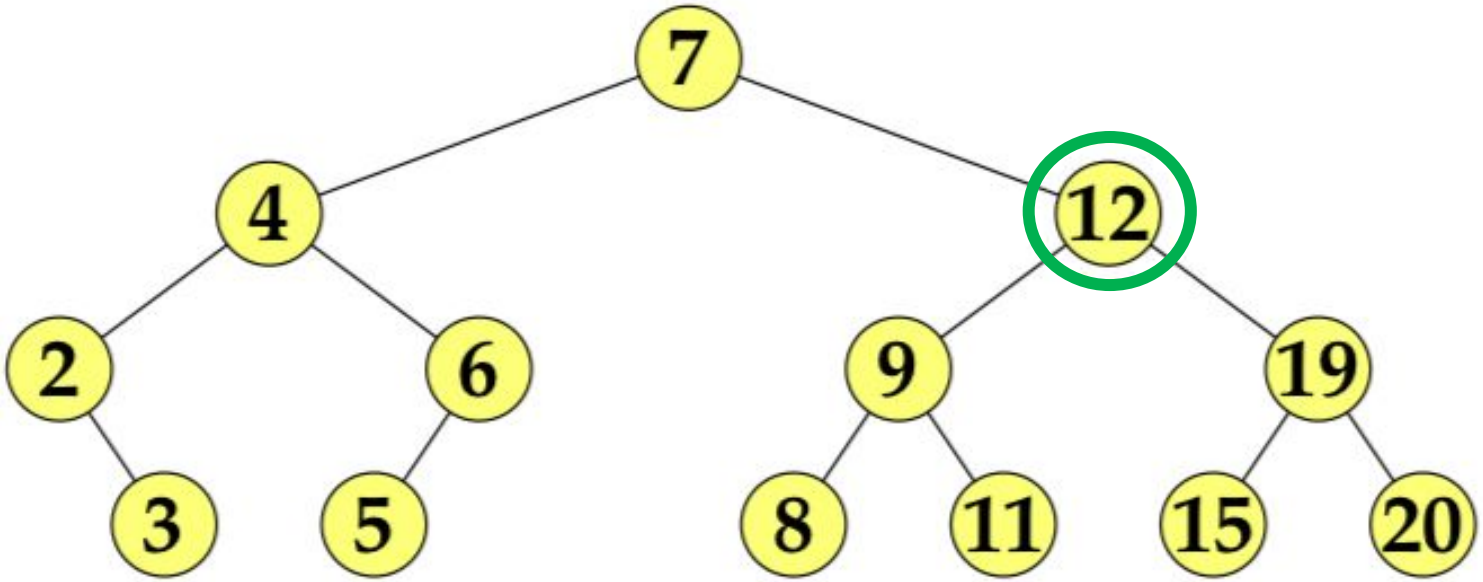


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7

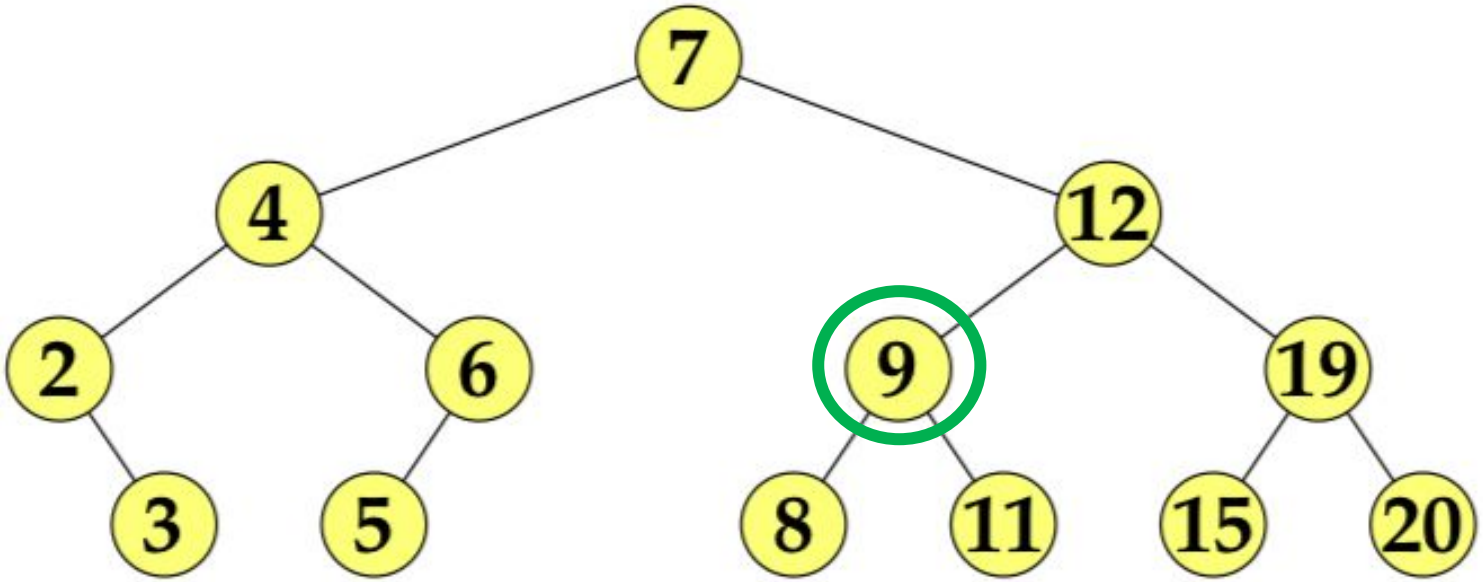


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7

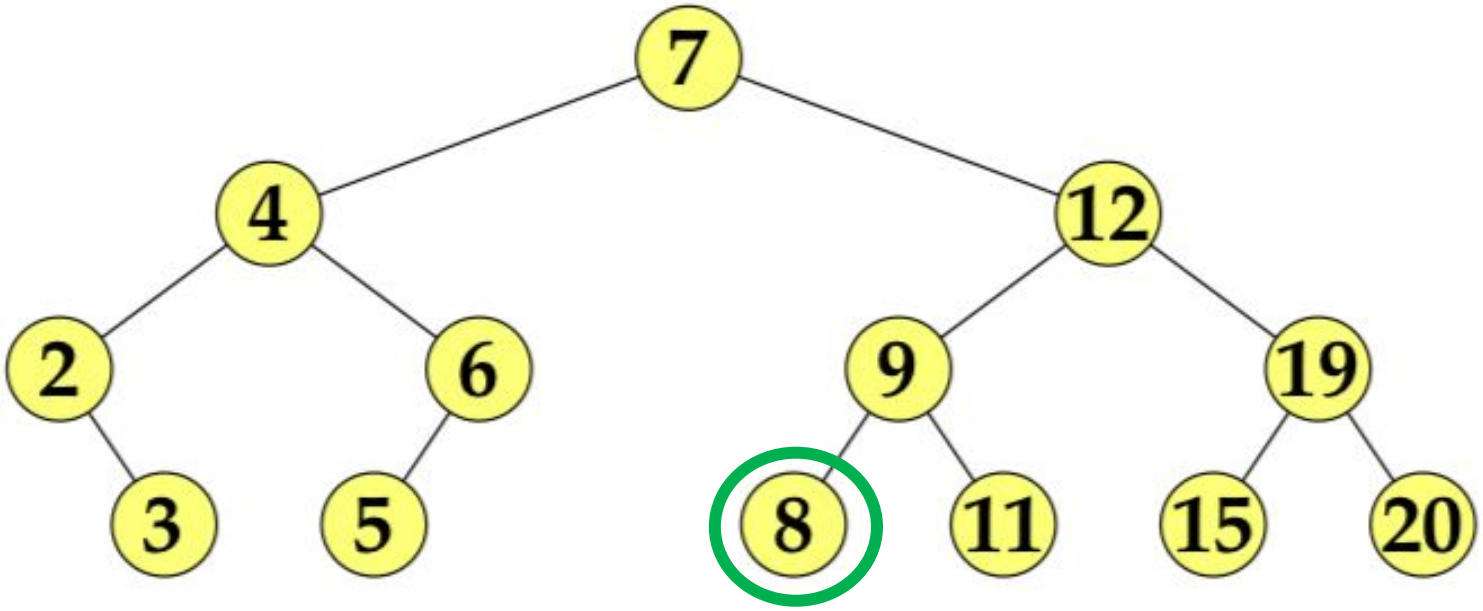


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7

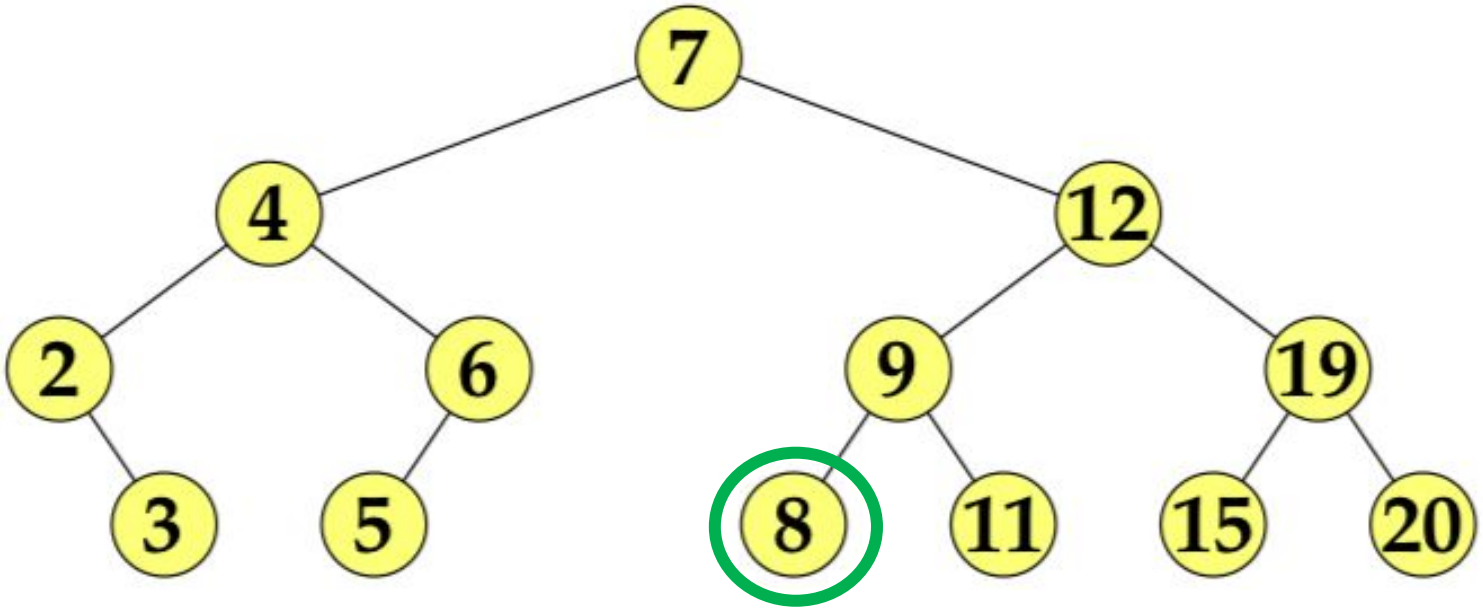


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8

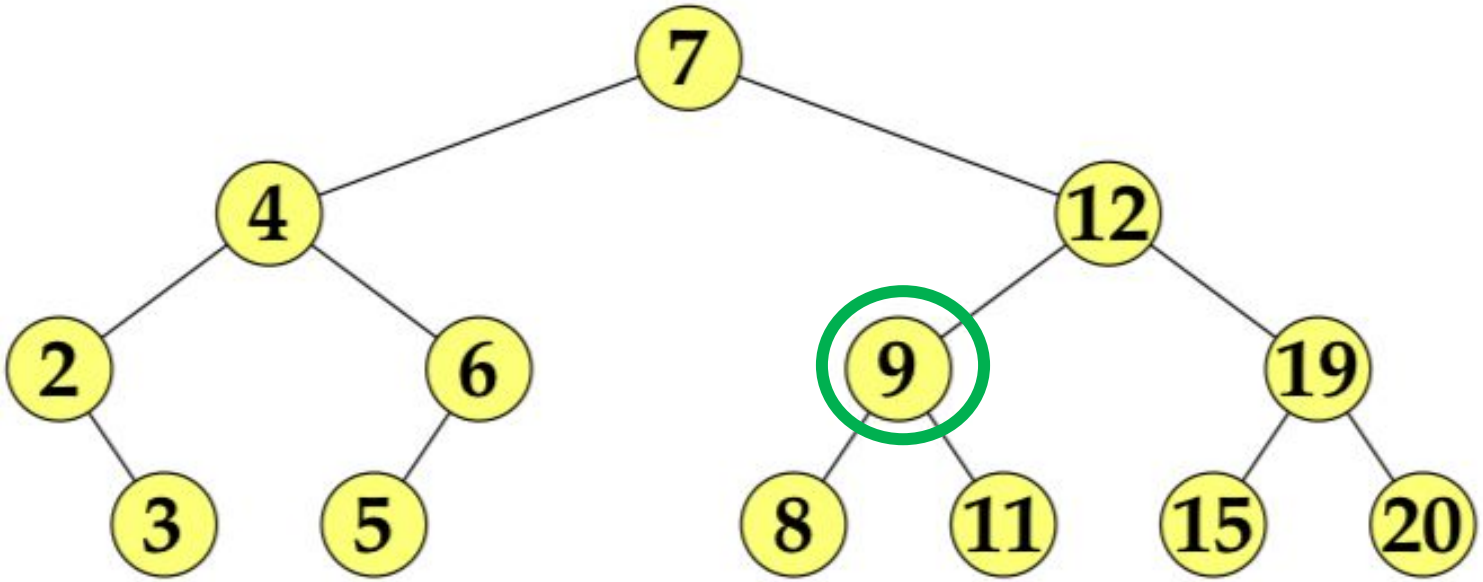


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8

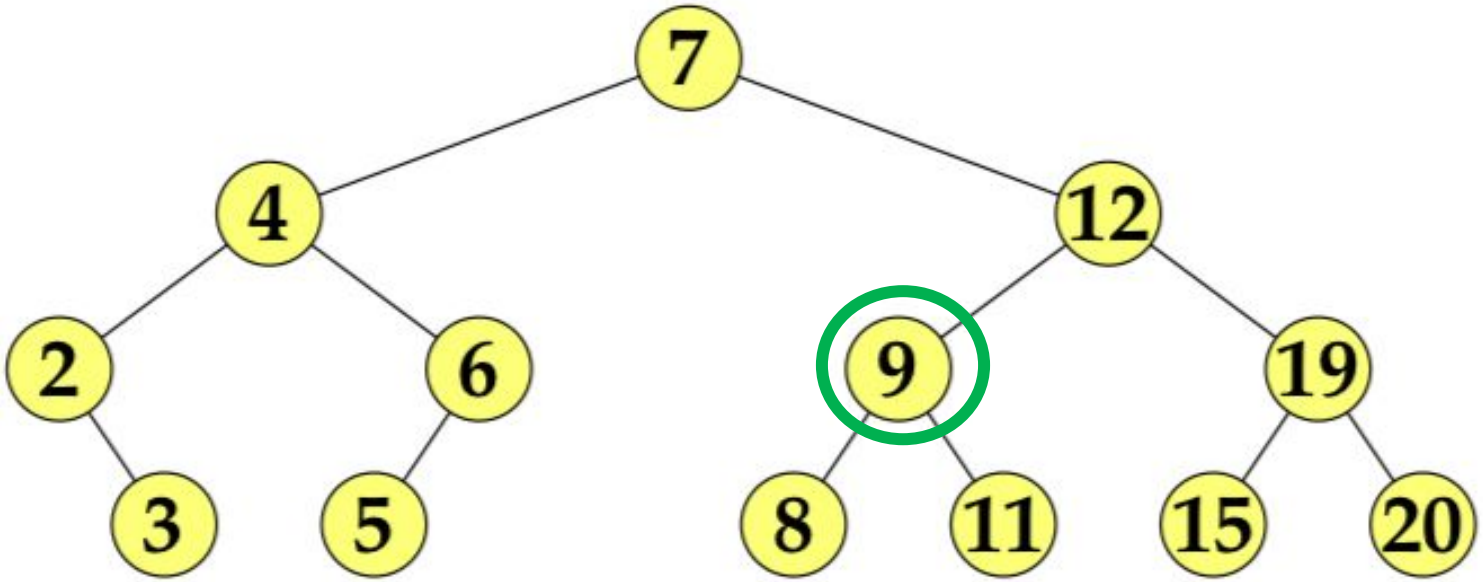


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9

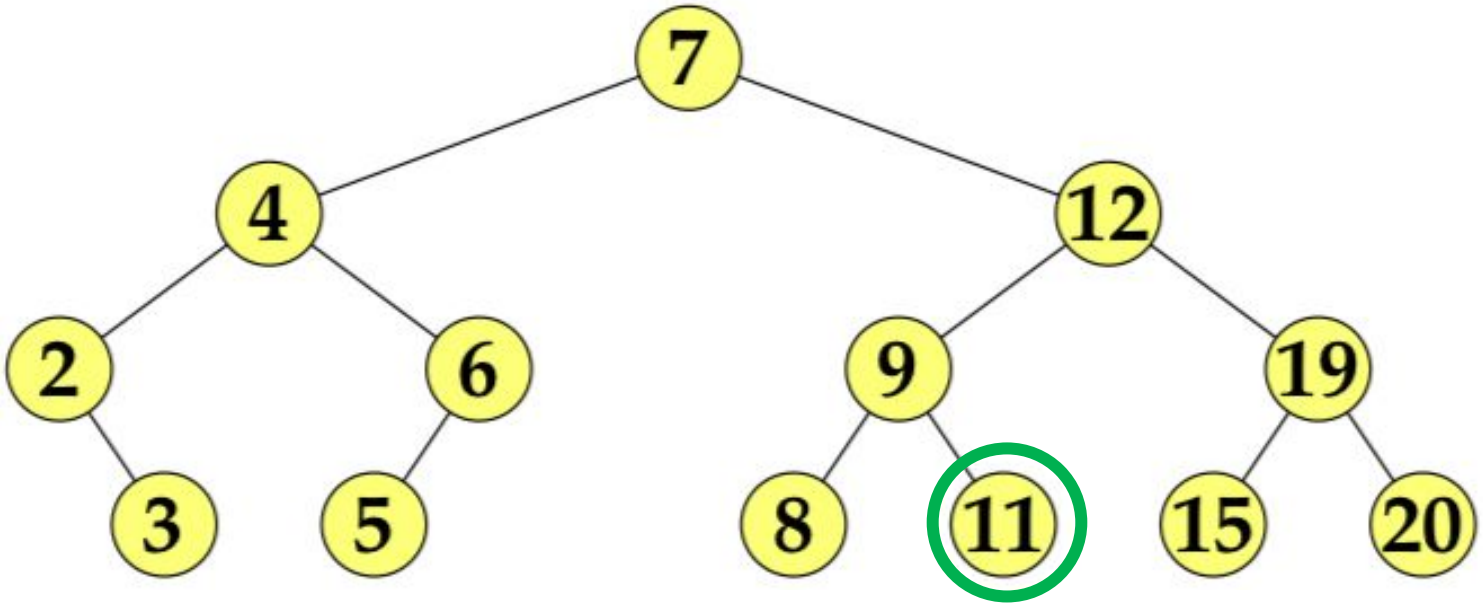


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9

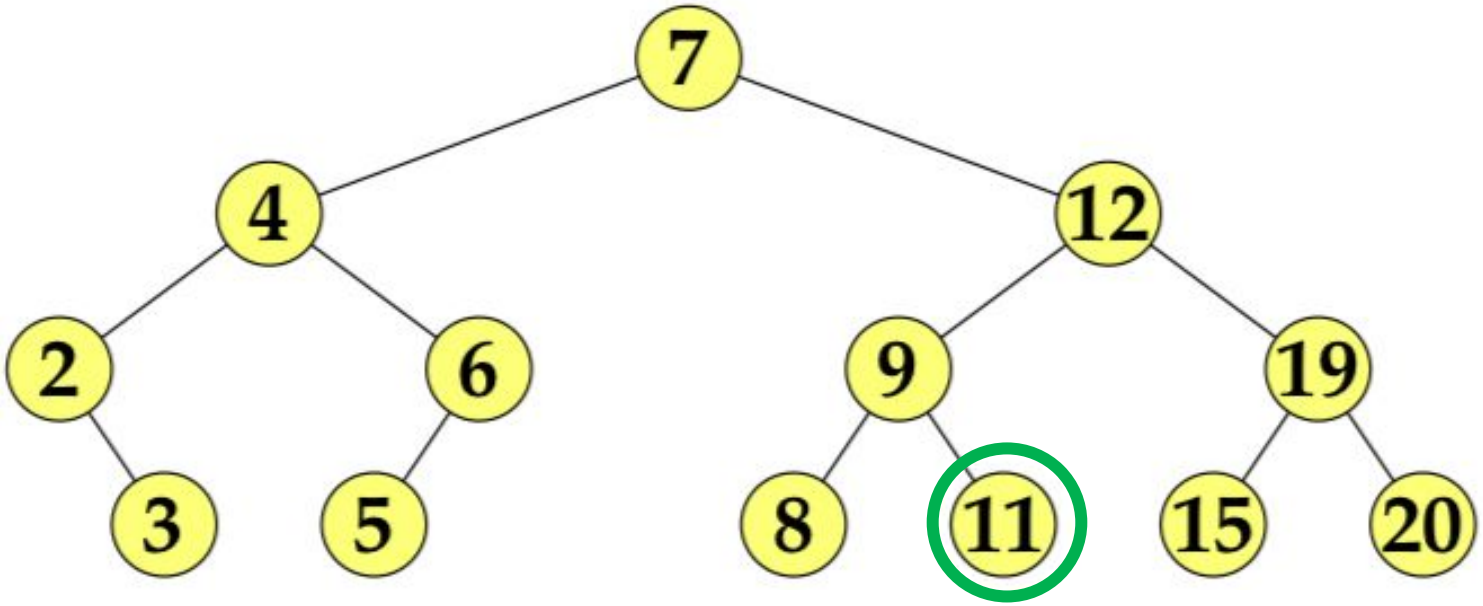


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11

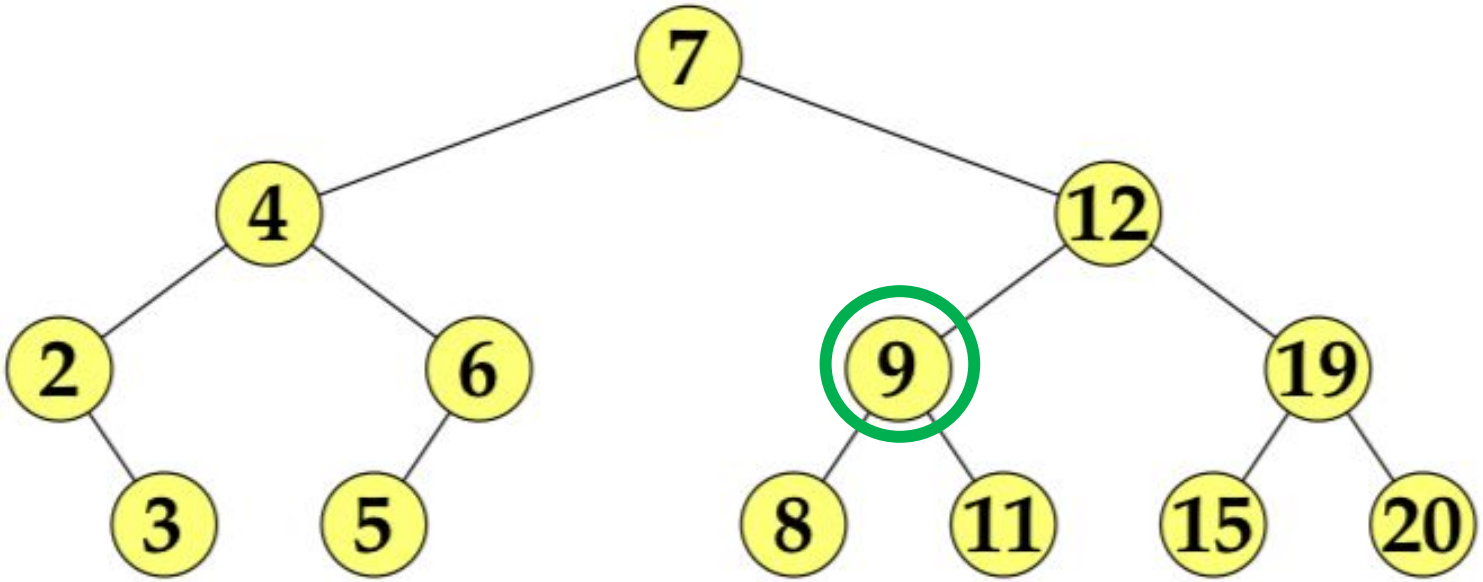


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11

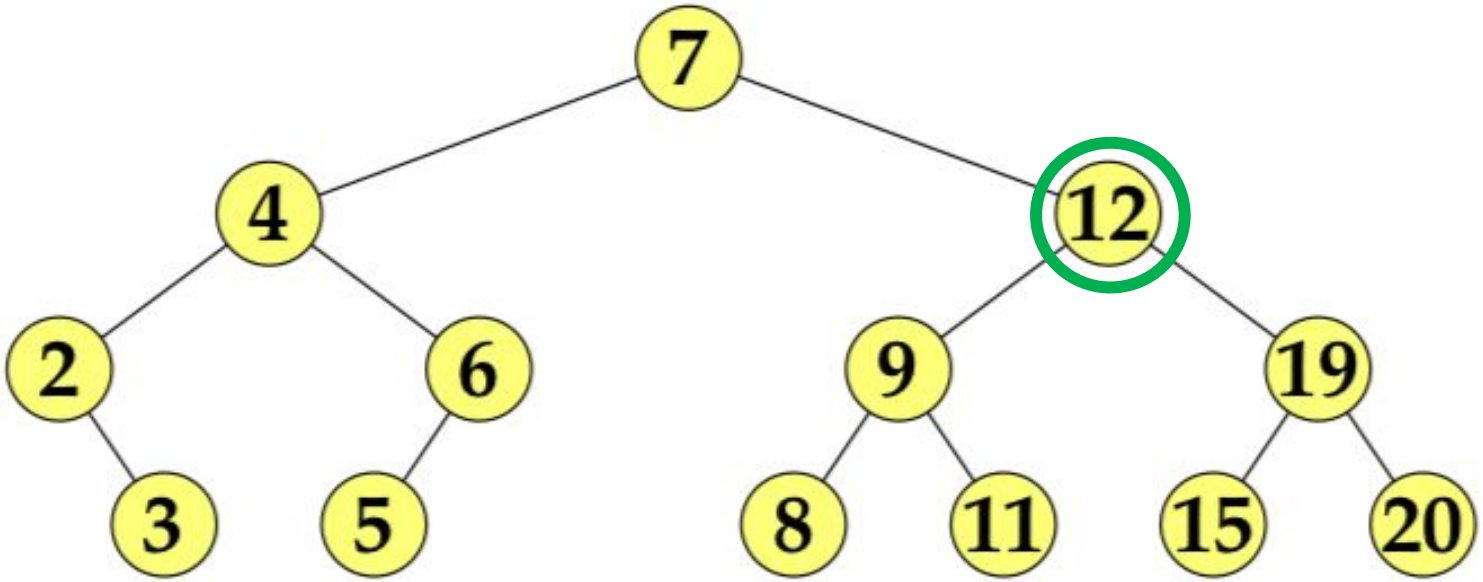


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11

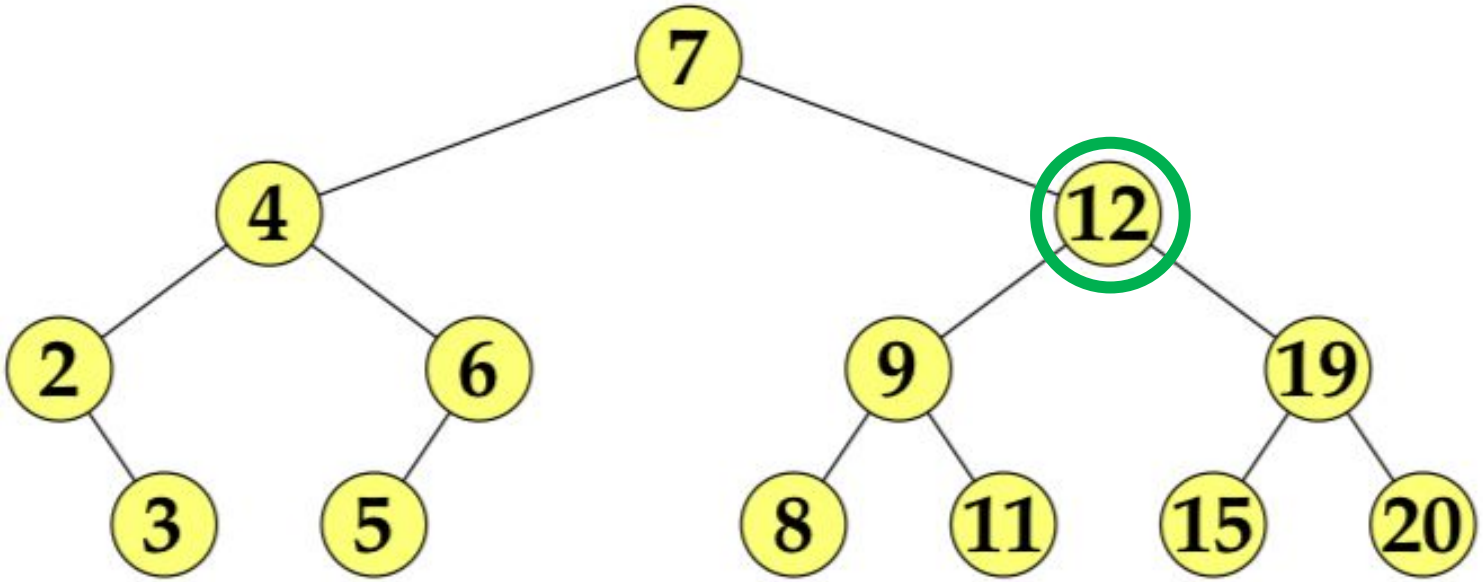


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12

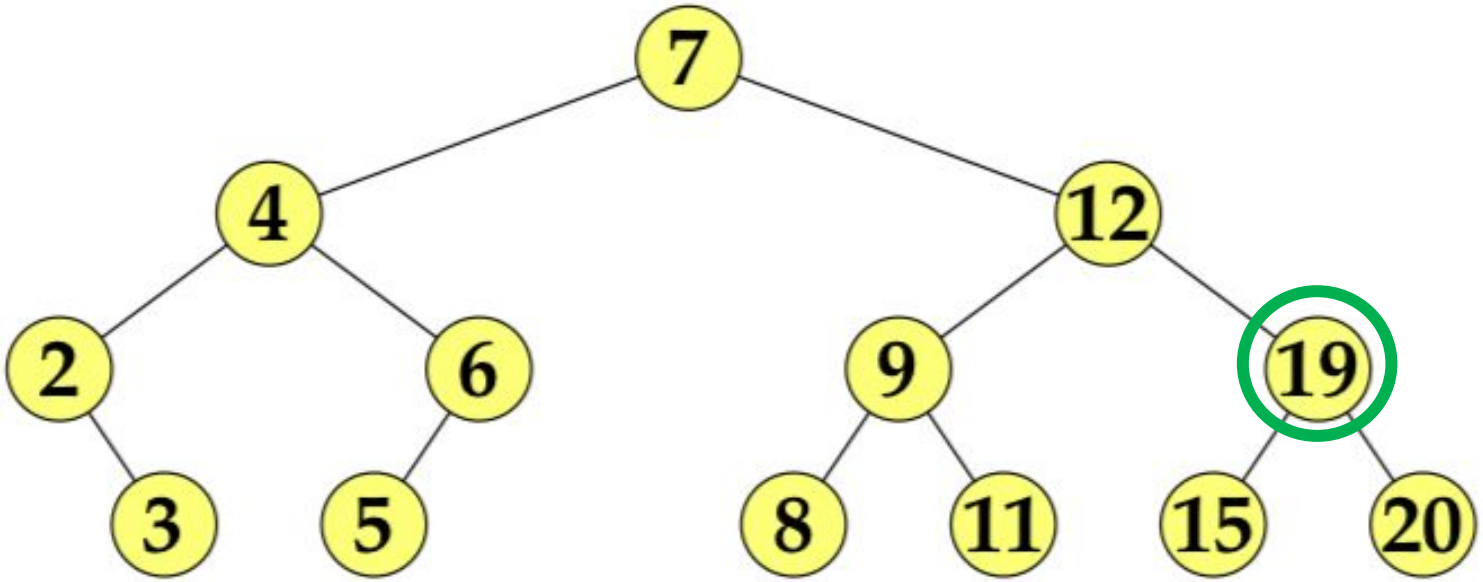


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12

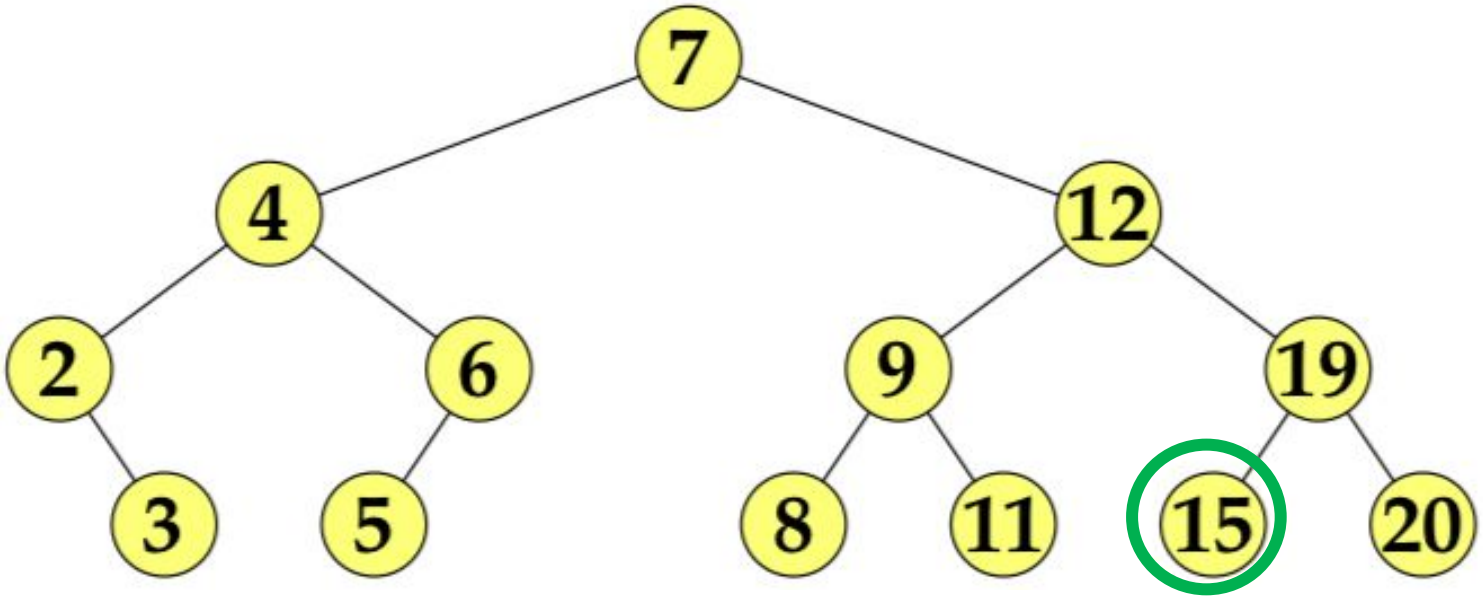


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12

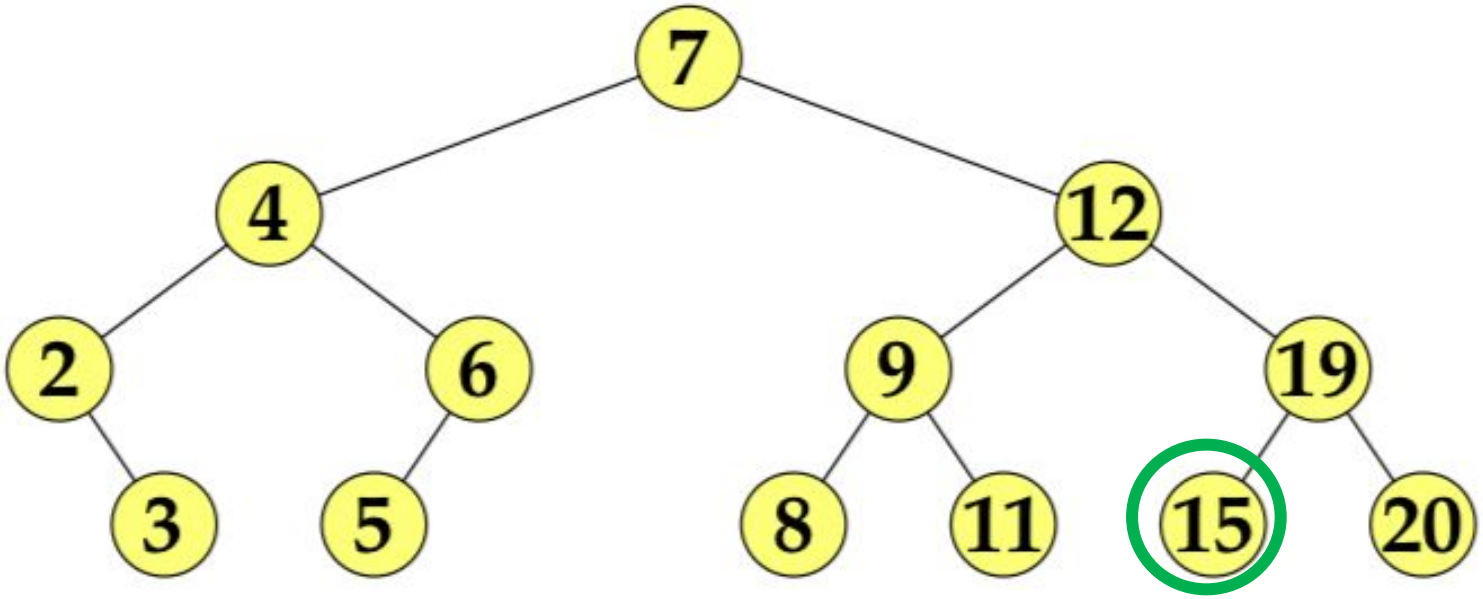


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15

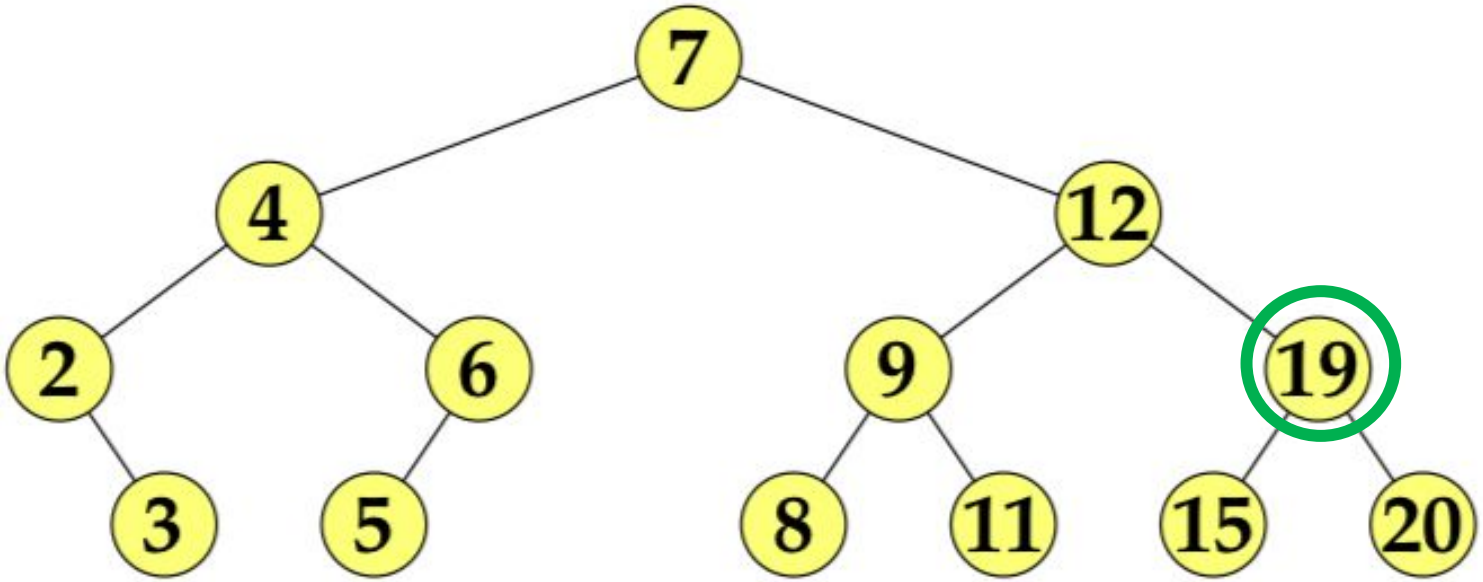


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15

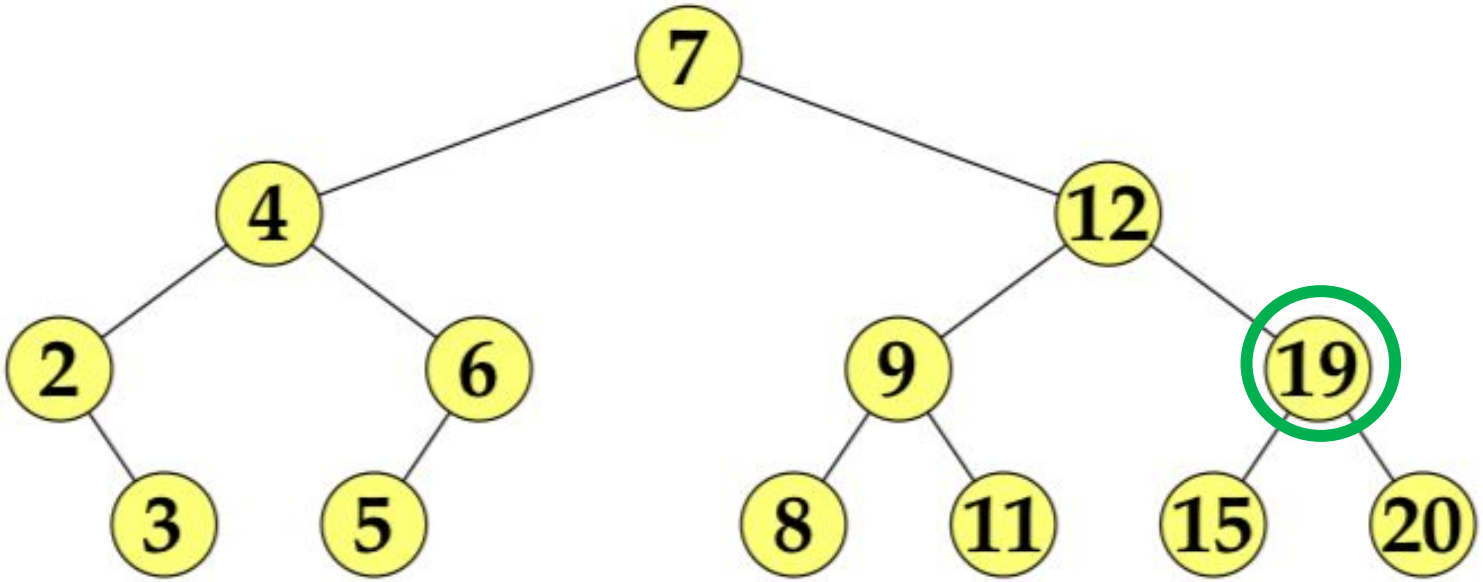


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, *19*

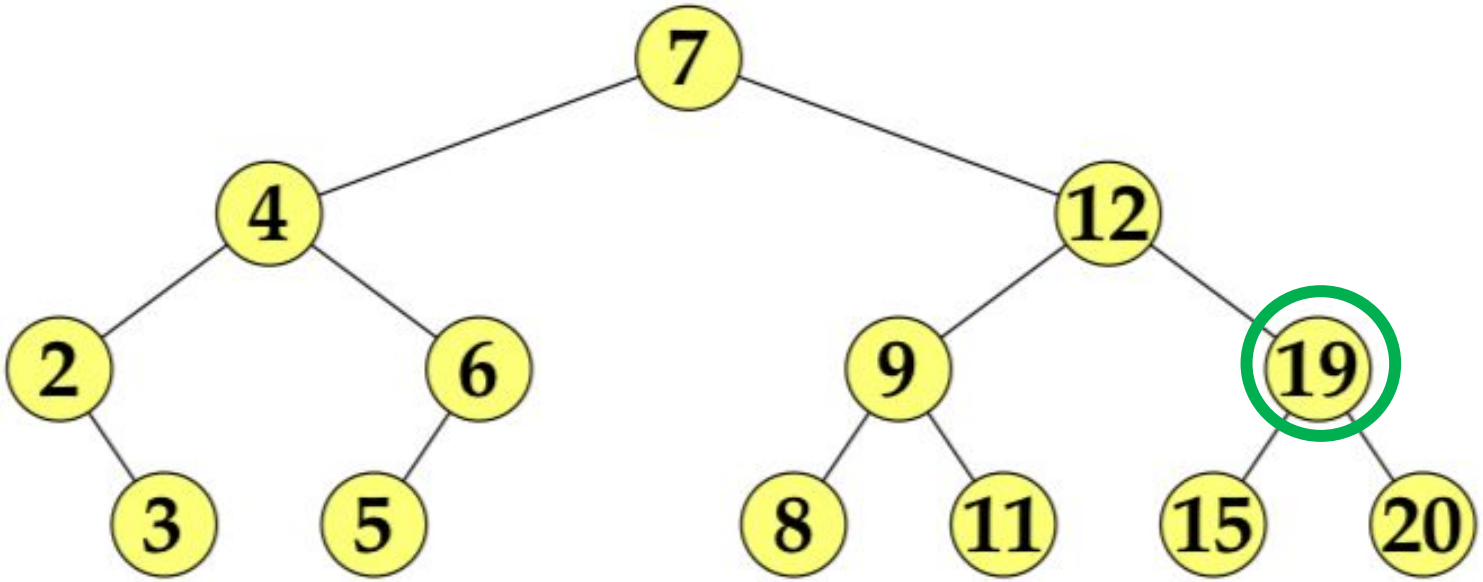


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19

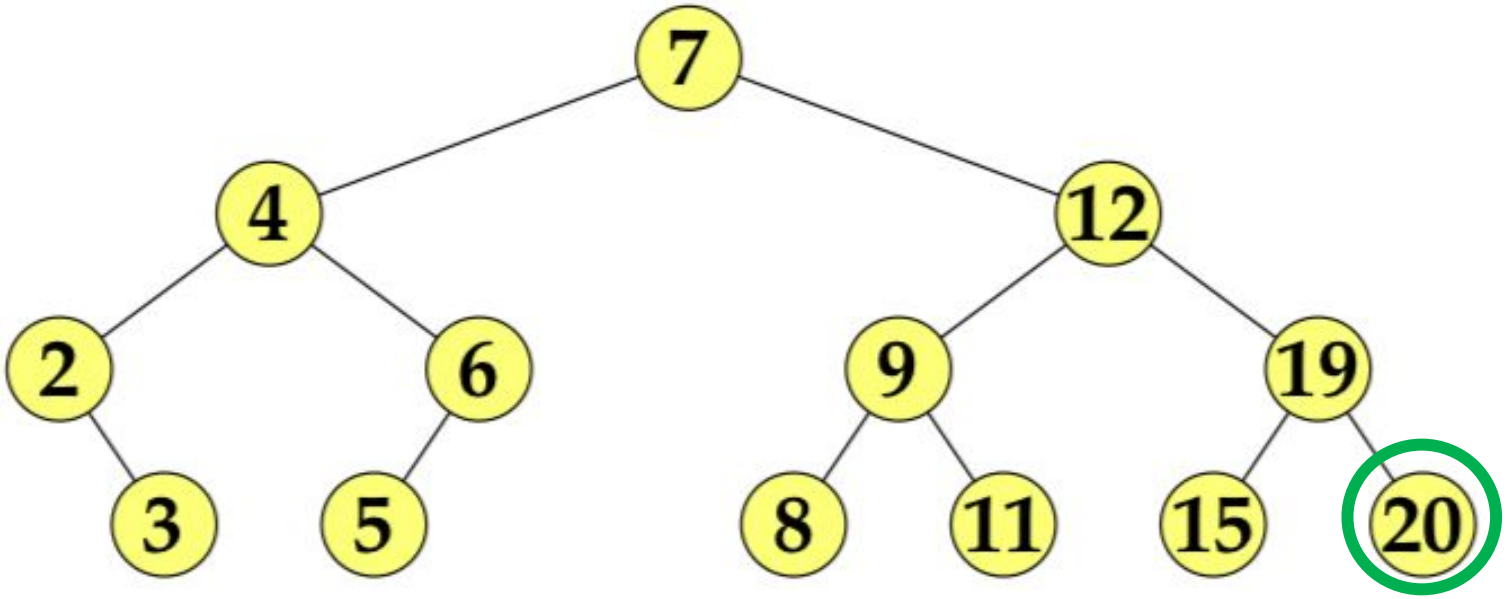


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19

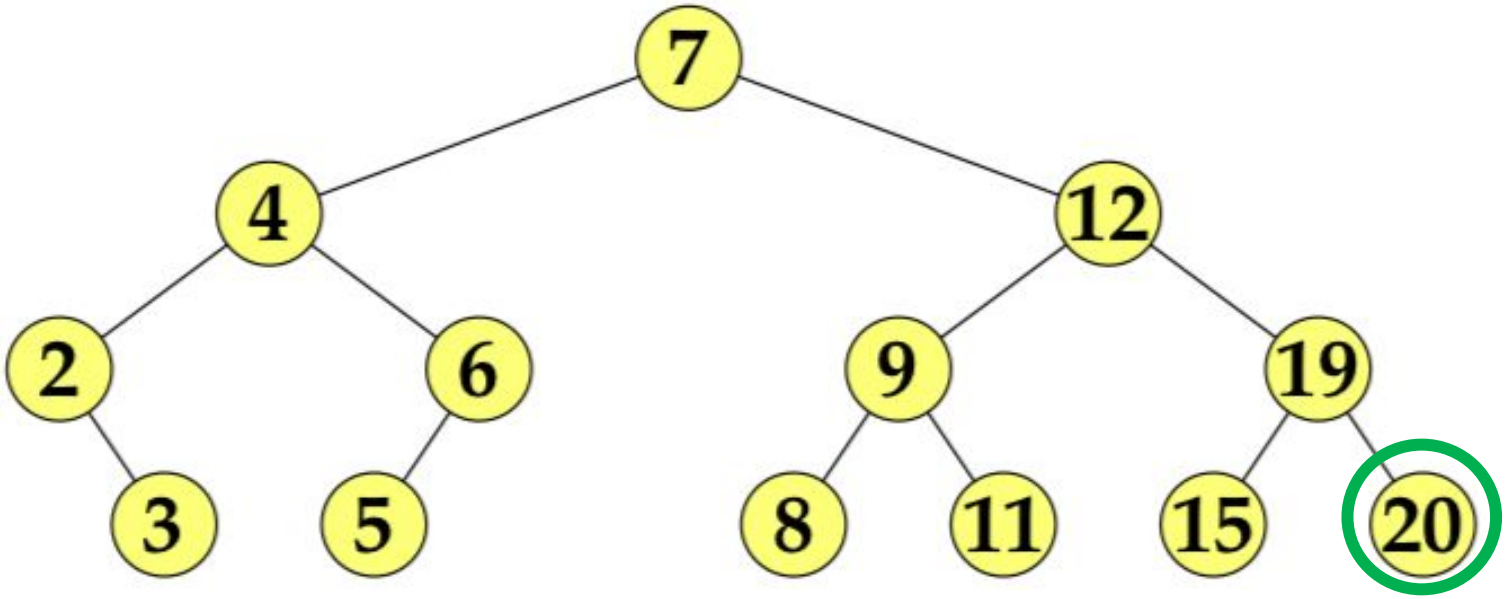


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19, 20

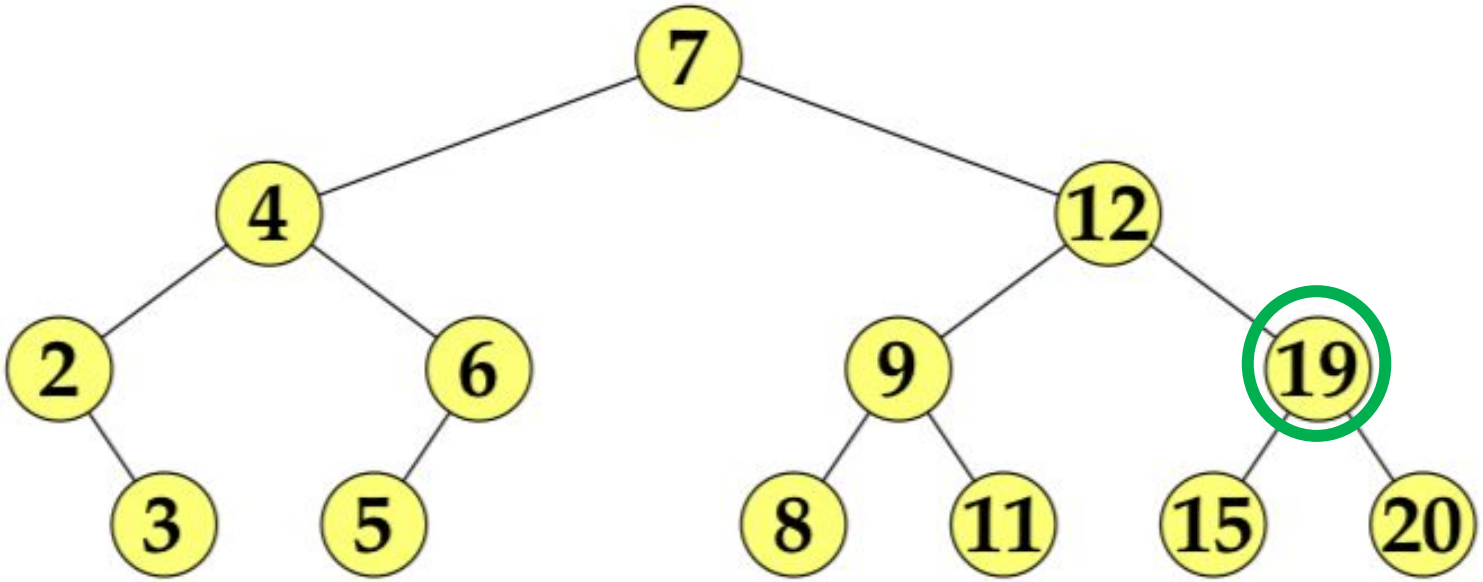


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19, 21

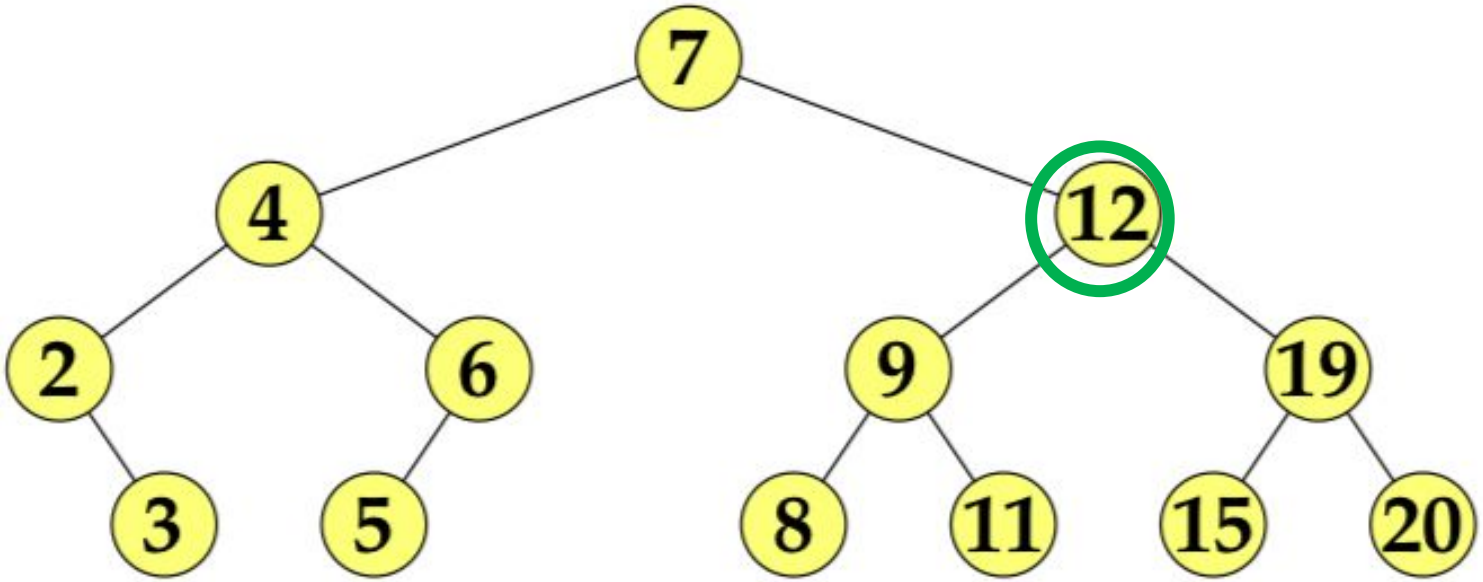


Inorder

What would the in-order traversal be here?

left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19, 21

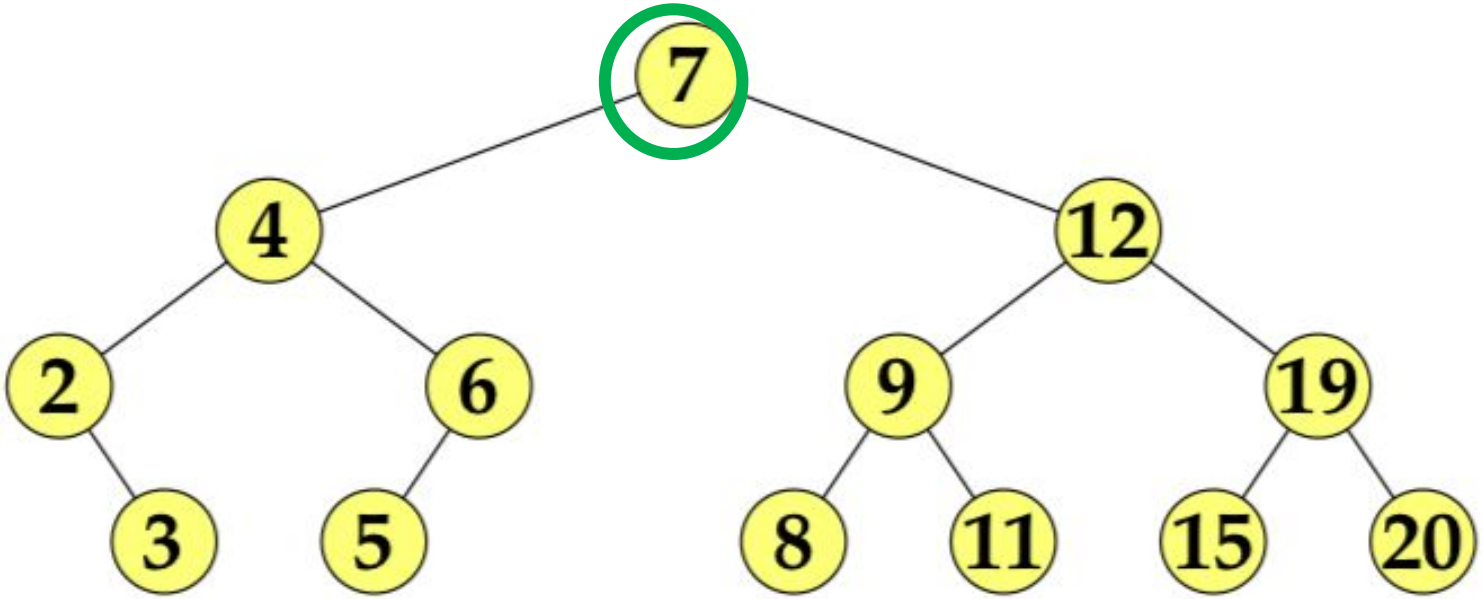


Inorder

What would the in-order traversal be here?

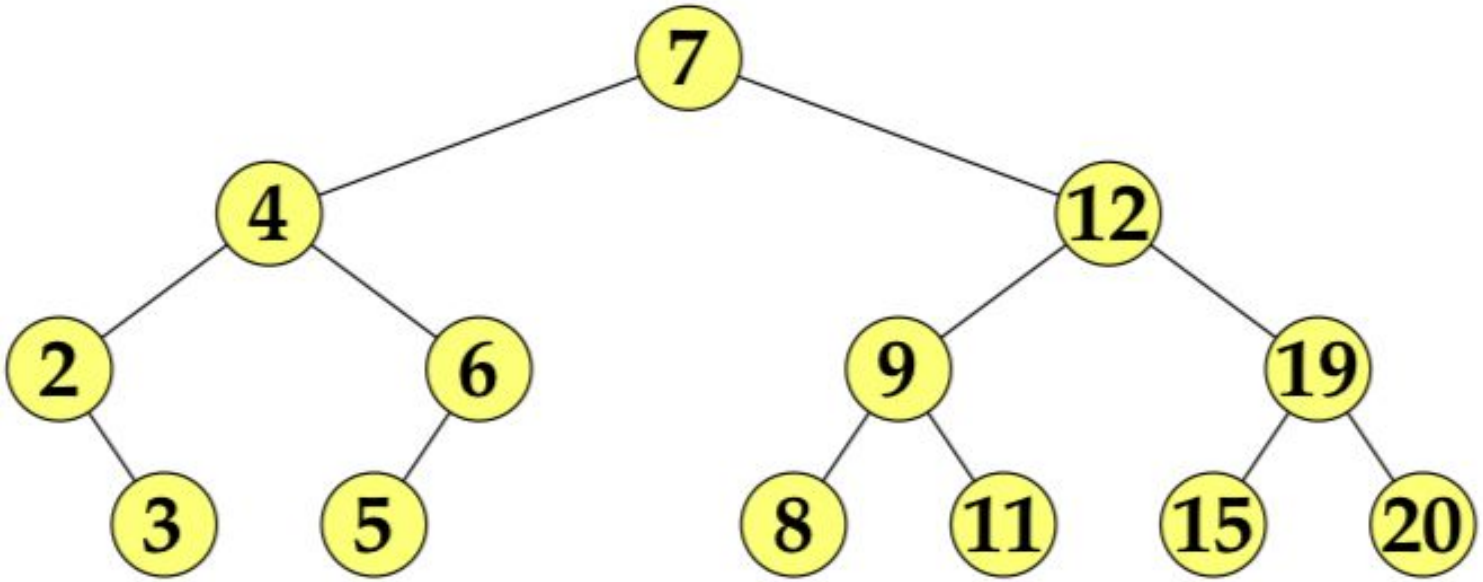
left subtree, current, right subtree

2, 3, 4, 5, 6, 7, 8, 9, 11, 12, 15, 19, 21



Inorder

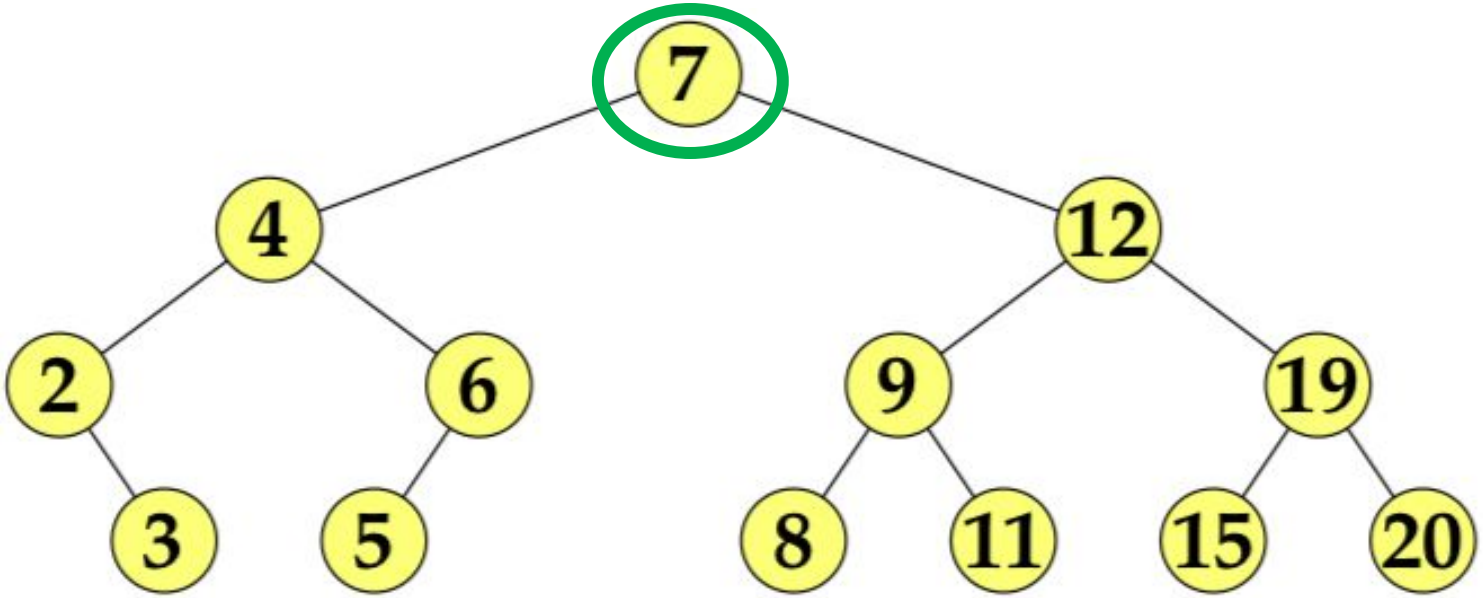
- 2, 3, 4, 5, 6, 7, 8 , 9, 11, 12, 15, 19, 20



preorder

What would the pre-order traversal be here?

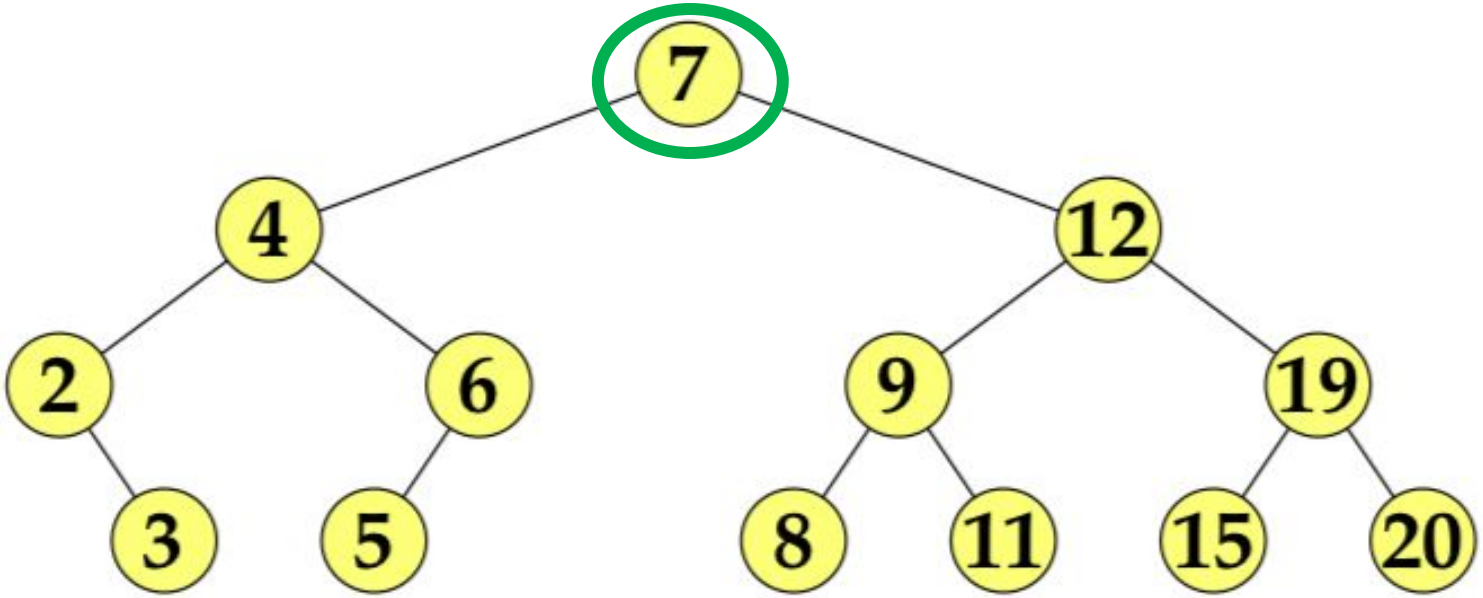
current,left subtree, right subtree



postorder

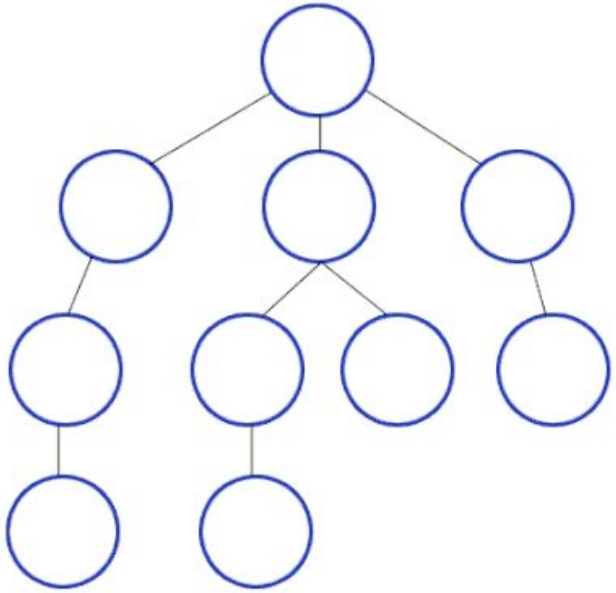
What would the post-order traversal be here?

left subtree, right subtree, current



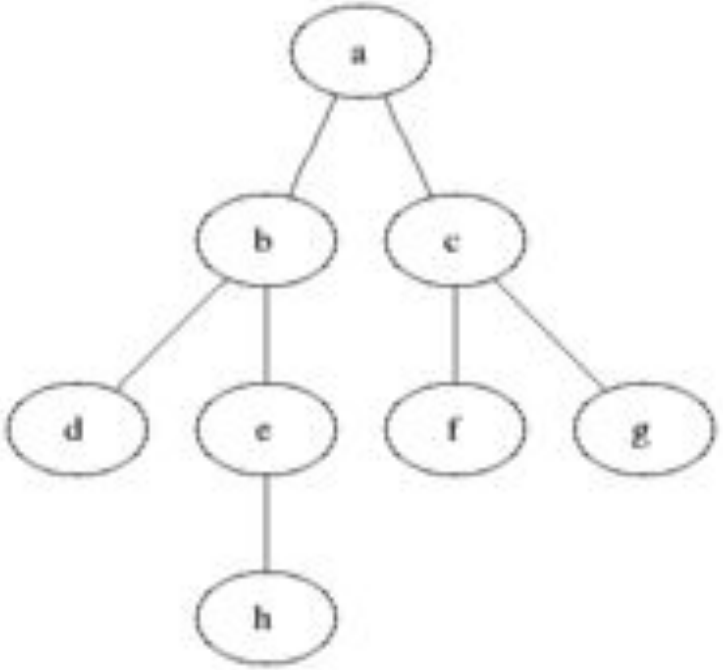
Depth First Search (DFS)

- Depth First Search (DFS)
  - start at root node and explore as far as possible along each branch
  - applications? when have we used this in class?



Breadth First Search (BFS)

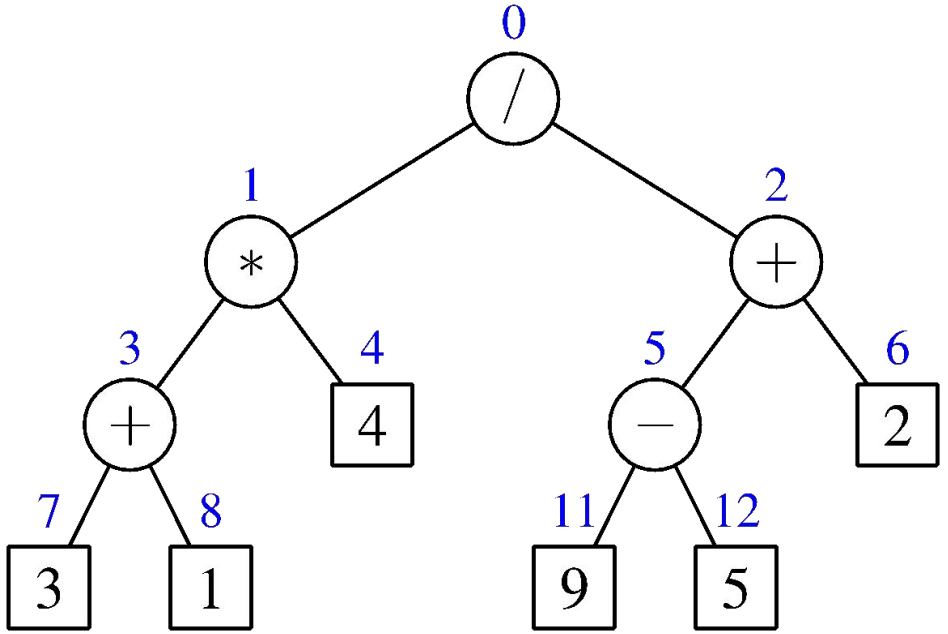
- Breadth First Search (BFS)
  - Starts at the root and explores all nodes at the present “depth” before moving to nodes on the next level
  - Extra memory is usually required to keep track of the nodes that have not yet been explored



Breadth-First Traversal

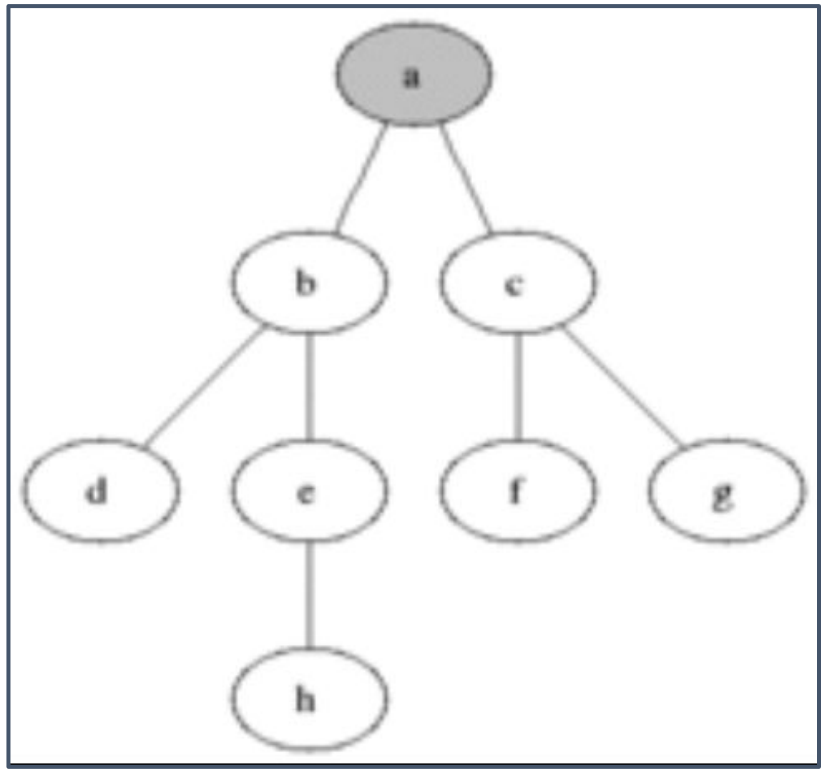
Traverse the tree level-by-level

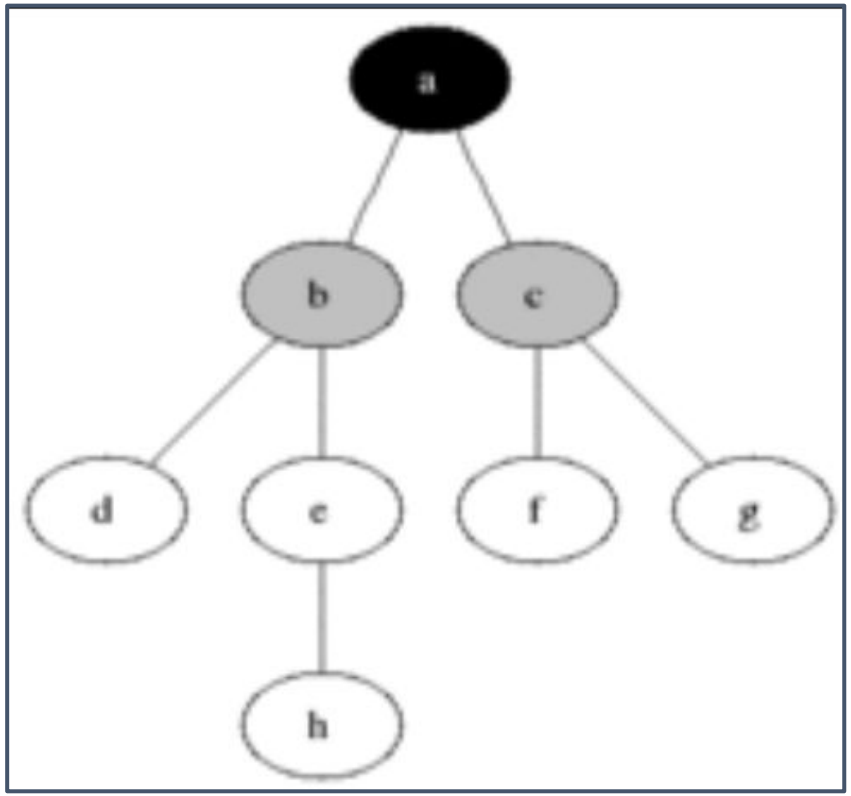
- Within a level go left-to-right

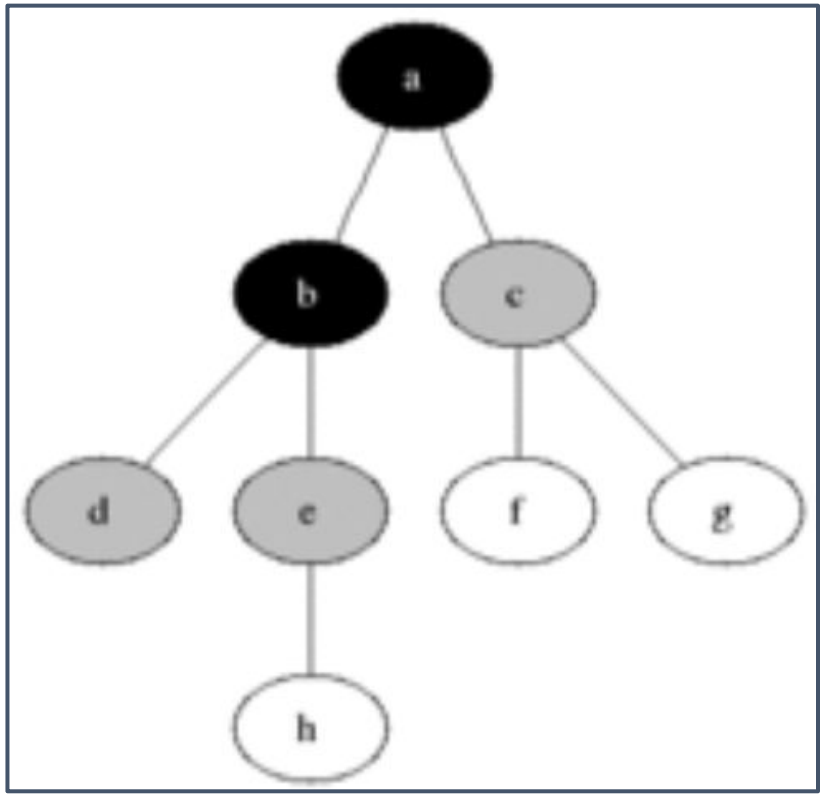


Breadth-First Traversal

pseudo-code?

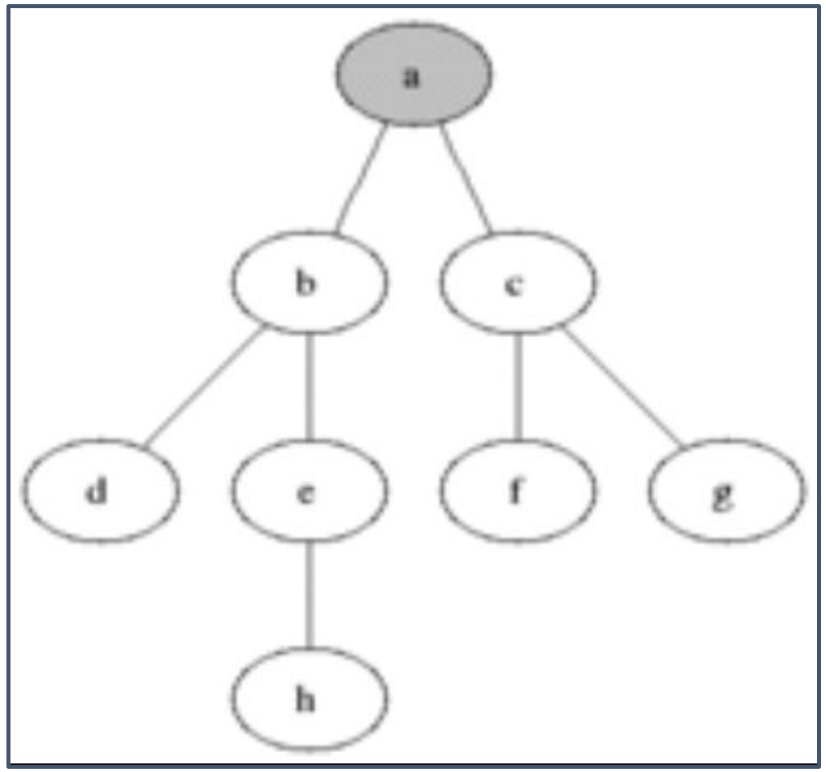


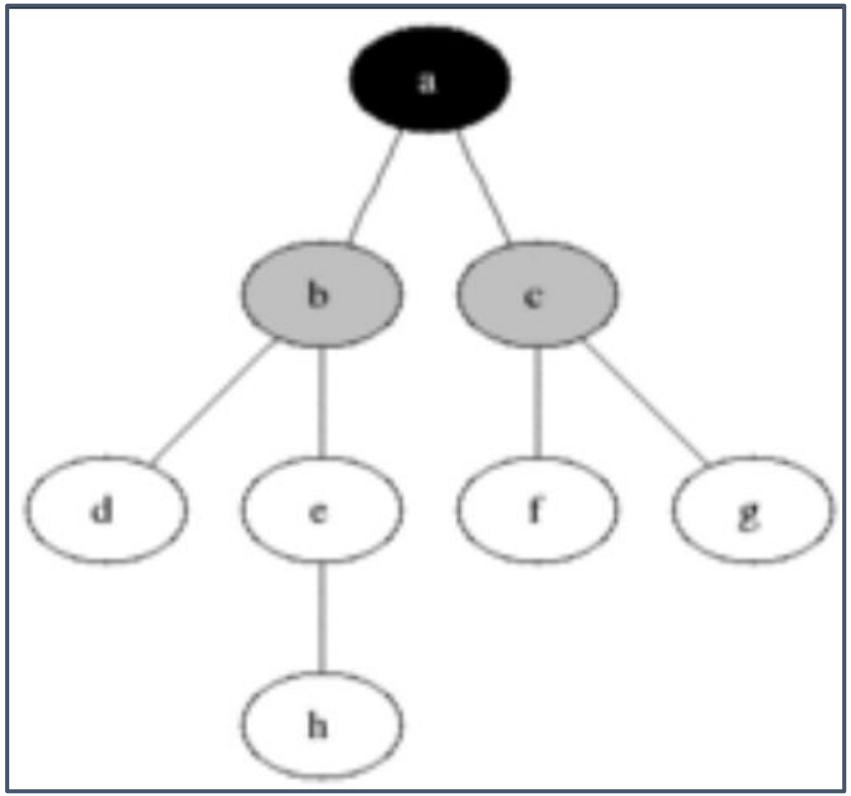


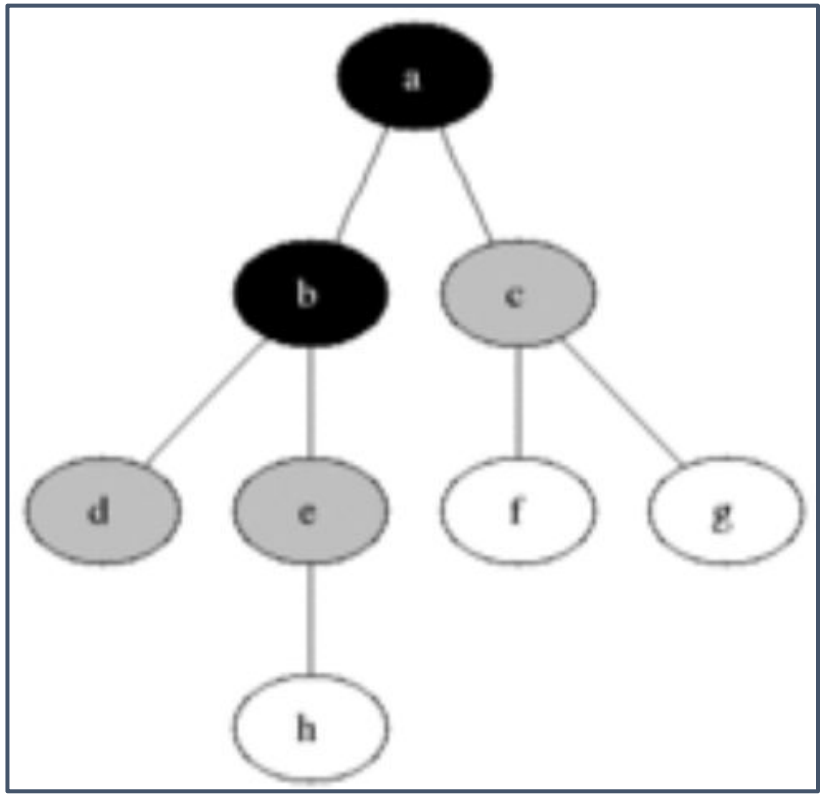


Breadth-First **Search**

How would we change the pseudo-code?



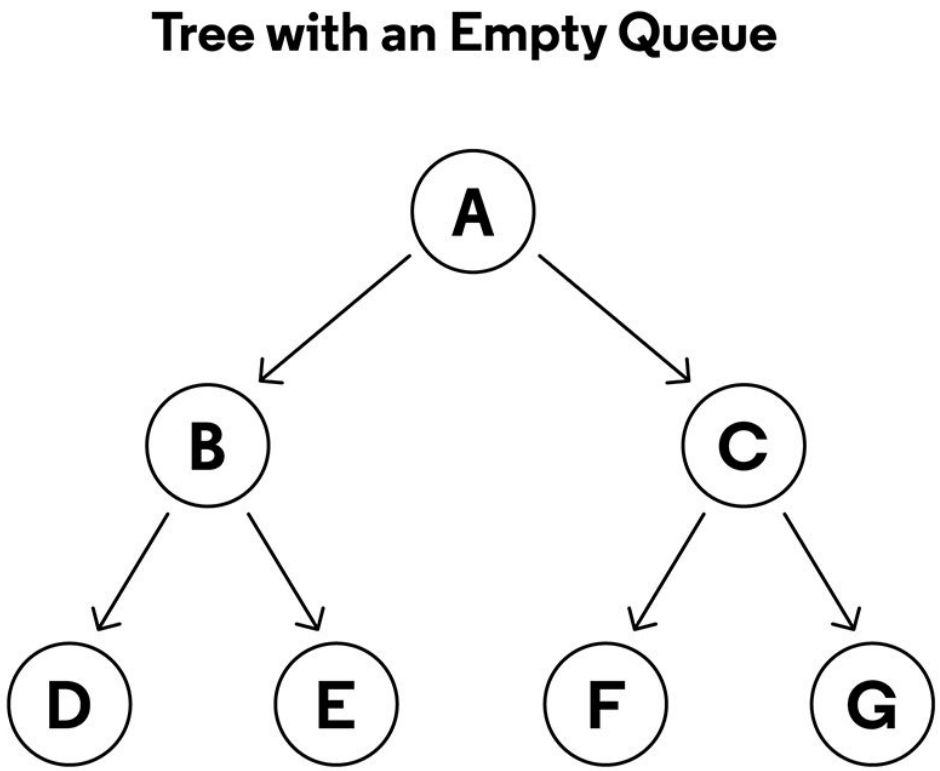




Breadth First Search (BFS)



Frontier Queue
FIFO (First in First Out)



https://www.codecademy.com/article/tree-traversal

Summary

Takeaways:

Binary search trees are an efficient data structure for search

For a *balanced* binary search tree:

- Search: O(log n)
- Insertion: O(log n)
- Removal: O(log n)